

المدرسة الوطنية للعلوم التطبيقية بتطوان
ⵜⵜⵉⵎⵔⵔⴰⵜ ⵜⴰⵏⴰⵎⵓⵔⵜ ⵏ ⵜⵙⵙⵏⴰⵢⵉⵏ ⵜⵉⵜⴰⵡⵉⵏⵉⵏ
Ecole Nationale des Sciences Appliquées de Tétouan

Titre de projet :

Détection de la Pneumonie à partir de Radiographies Thoraciques à l'aide de l'Apprentissage Profond

École nationale des sciences appliquées de Tétouan

Filière : Science des Données, Big Data et IA

Département : Intelligence Artificielle et Digitalisation (IAD)

Encadré par :

Imane HACHCHANE : Professeure d'apprentissage automatique

Réalisé par :

Yassir Adila *(Inception)*
Apogée : 22048197
Mohamed Channa *(Custom CNN)*
Apogée : 22053046
Aymane Azaagag *(VGG)*
Apogée : 22049082
Mohamed Abrini *(DenseNet)*
Apogée : 22043791
Aymane Hayoun *(ResNet)*
Apogée : 22049899

Table des matières

1 Introduction & Présentation de la base de données **4**

2 Modèle Inception **6**
2.1 Introduction à l'Architecture Inception 6
2.2 Contexte et Problématique 6
2.3 Principes Fondamentaux du Module Inception 7
2.4 Évolution de l'Architecture : De Inception V1 à V3 9
2.5 Architecture et Innovations d'Inception V3 10
2.5.1 Factorisation des Convolutions 10
2.5.2 Factorisation Asymétrique 10
2.5.3 Normalisation par Lots (Batch Normalization) 11
2.5.4 Réduction Intelligente des Cartes de Caractéristiques 13
2.5.5 Classificateurs Auxiliaires et Régularisation 14
2.6 Architecture Inception : Structure et Organisation des Modules Convolutifs . 15
2.6.1 Couches initiales 16
2.6.2 Modules Inception : principes et variantes 16
2.6.3 Fusion des branches 17
2.6.4 Étapes finales 17
2.7 Avantages et Limites d'Inception V3 17
2.8 Conclusion 18

3 Modèle ResNet **19**
3.1 Présentation de ResNet et motivation 19
3.2 Architecture de ResNet-50 20
3.3 Fonctionnement et particularités techniques 22
3.4 Applications et performances 24
3.5 Limites de ResNet 25
3.6 Évolutions de ResNet 26
3.7 Conclusion 28

4 Modèle DenseNet **29**
4.1 Introduction 29
4.2 Problématique du diagnostic médical automatisé 29
4.3 Architecture du Modèle DenseNet101 29
4.4 Représentation de l'architecture 30
4.5 Préparation des Données et Entraînement 31
4.6 Limites et inconvénients de DenseNet-101 31
4.7 Conclusion 32

5 Modèle VGG **33**
5.1 Introduction 33
5.2 Présentation du modèle VGG16 33
5.2.1 Origine du modèle VGG 33

5.2.2 Structure et architecture du modèle VGG16 . . . 34
5.2.3 Comparaison entre les architectures VGG16 et VGG19 . . . 36
5.2.4 Aspects mathématiques de VGG16 . . . 37
5.2.5 Méthodologie . . . 39
5.2.6 Analyse critique du modèle VGG16 . . . 41
5.3 Conclusion . . . 42

6 Custom CNN **43**
6.1 Introduction . . . 43
6.2 Contexte et Problématique . . . 43
6.3 Principes Fondamentaux du CNN . . . 43
6.4 Architecture du CNN Personnalisé . . . 44
6.5 Implémentation Complète . . . 44
6.5.1 Illustration : Architecture Complète du CNN Personnalisé . . . 47
6.5.2 Illustration : Zoom sur un Bloc Convolutif . . . 48
6.6 Techniques Clés Utilisées . . . 49
6.6.1 Augmentation des Données . . . 49
6.6.2 Normalisation par Lots . . . 49
6.6.3 Dropout . . . 49
6.6.4 Optimisation . . . 49
6.7 Hyperparamètres . . . 50
6.8 Évaluation des Résultats . . . 50
6.8.1 Analyse des Performances . . . 50
6.9 Limites du Modèle . . . 50
6.10 Discussion . . . 51
6.11 Perspectives d'Amélioration . . . 51
6.12 Conclusion . . . 51

7 Evaluation des modèles **53**
7.1 Inception V3 . . . 53
7.2 Custom CNN . . . 56
7.3 ResNet . . . 59
7.4 VGG-16 . . . 61
7.4.1 Evaluation sur l'ensemble de données d'entrainement et de validation 61
7.4.2 Évaluation sur l'Ensemble de Test . . . 63
7.5 DenseNet . . . 68

8 Comparaison des modèle **69**

9 Références bibliographiques **70**
9.1 Inception . . . 70
9.2 Custom CNN . . . 70
9.3 VGG . . . 71

1 Introduction & Présentation de la base de données

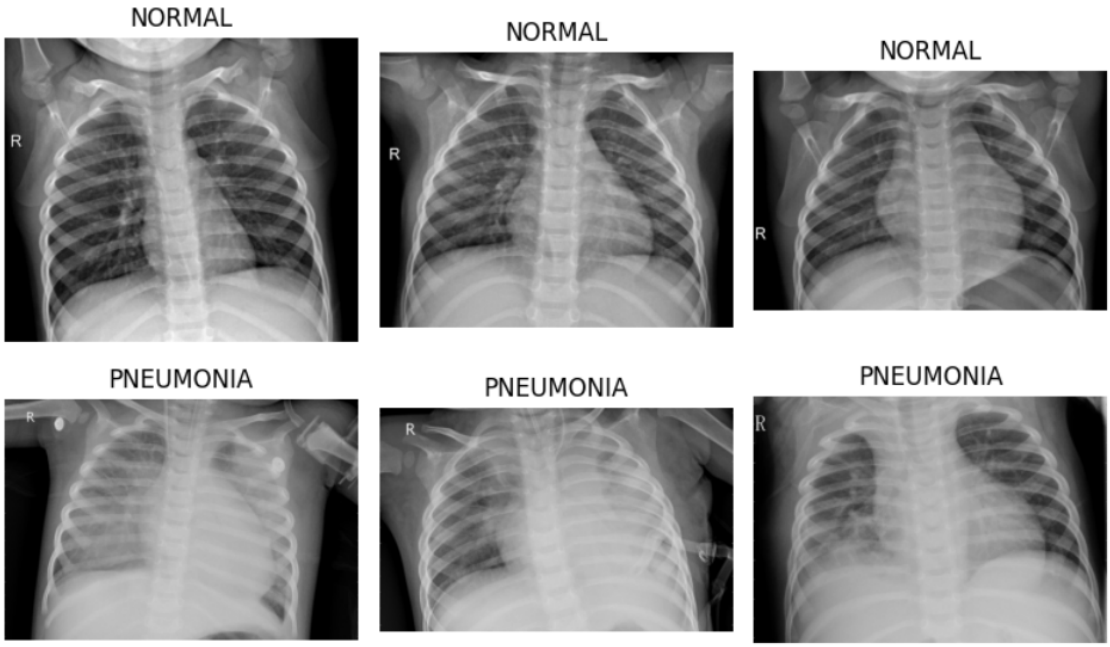
Dans le cadre de ce projet, nous nous sommes intéressés à la classification d'images médicales, en particulier à la détection automatisée de la pneumonie à partir de radiographies thoraciques. Ce sujet s'inscrit dans une problématique médicale réelle, dans laquelle l'intelligence artificielle peut jouer un rôle crucial pour assister les professionnels de santé dans la détection rapide et fiable de pathologies pulmonaires.

Pour mener cette étude, nous avons utilisé la base de données publique intitulée **"Chest X-Ray Images (Pneumonia)"**, disponible sur la plateforme Kaggle **(www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia)** et initialement publiée par Paul Timothy Mooney. Cette base de données constitue une référence dans les travaux de recherche sur la classification de la pneumonie à partir d'images radiographiques.

Elle contient au total 5 863 images de radiographies thoraciques (au format JPEG), classées en deux catégories :

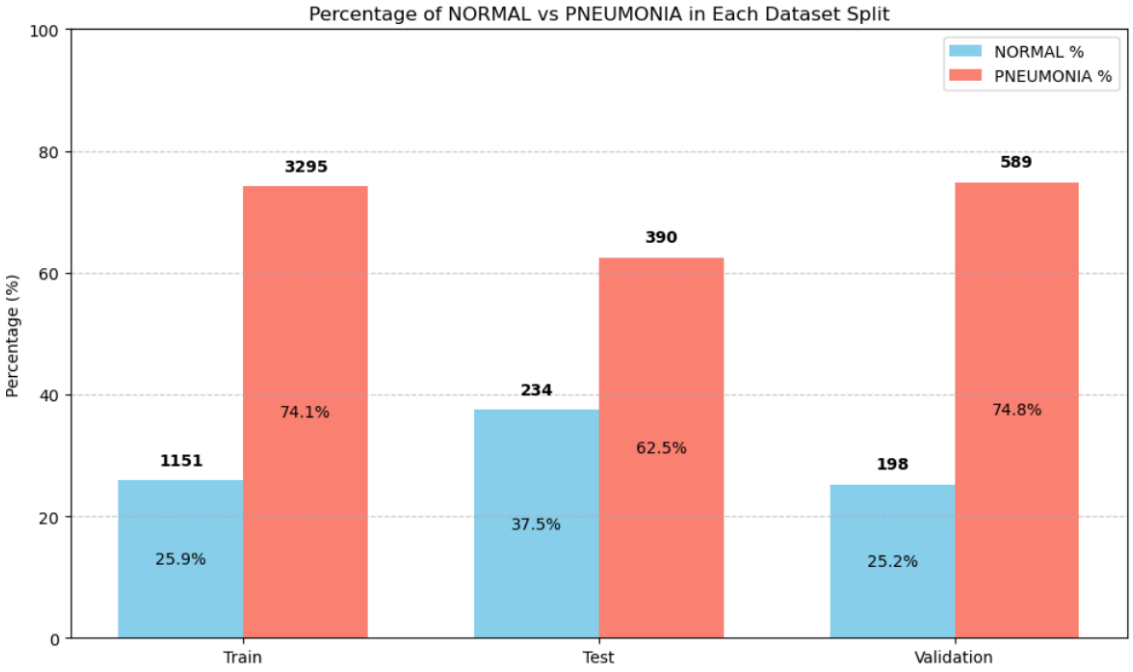
— "Normal" : radiographies de patients sains,

— "Pneumonia" : radiographies de patients diagnostiqués avec une pneumonie (bactérienne ou virale).



Les images sont organisées en trois dossiers distincts : train, val, et test, afin de faciliter l'entraînement, la validation et l'évaluation des modèles. Cette organisation permet de simuler un environnement réaliste de développement d'un modèle de deep learning.

Cependant, nous avons constaté un déséquilibre notable dans la répartition du nombre d'images, en particulier dans le dossier de validation, qui ne contenait initialement que 16 images. Un tel volume s'est révélé insuffisant pour permettre une évaluation fiable des performances du modèle pendant l'entraînement.
Pour remédier à cette limitation, nous avons procédé à un rééchantillonnage manuel : une portion représentative des données du dossier train a été extraite et redirigée vers le dossier val, de manière à constituer un jeu de validation plus robuste et statistiquement significatif.



Répartition finale des données après réajustement du jeu de validation

D'un point de vue visuel, les radiographies de la classe Pneumonia présentent souvent des opacités anormales au niveau des poumons, visibles sous forme de zones blanchâtres diffuses, contrairement aux poumons clairs observés dans les images Normales.

Ainsi, la nature binaire de la classification, la qualité et la disponibilité des données, ainsi que la pertinence médicale du sujet, font de cette base de données un excellent support pour appliquer des modèles d'apprentissage profond, notamment les réseaux de neurones convolutifs (CNN).

2 Modèle Inception

2.1 Introduction à l'Architecture Inception

Les réseaux de neurones convolutifs (CNN) ont transformé la vision par ordinateur, notamment pour la classification d'images. Des architectures telles que LeNet, AlexNet et VGG ont marqué des étapes importantes, mais ont rapidement montré leurs limites : trop de couches, trop de paramètres, et un coût computationnel élevé. Ces contraintes rendent leur déploiement difficile, surtout dans des contextes à ressources limitées.

Pour répondre à ces enjeux, Google a proposé en 2014 l'architecture Inception, introduite lors de la compétition ImageNet. Elle innove par l'usage de modules multi-branches, la factorisation des convolutions, et la réduction de la dimensionnalité. Ces choix permettent de bâtir des réseaux plus profonds et efficaces, tout en maîtrisant les ressources. Les versions ultérieures, jusqu'à Inception V4, ont successivement amélioré ces principes. Parmi elles, Inception V3 se distingue particulièrement par ses optimisations avancées, et constitue le modèle utilisé dans notre projet de détection de pneumonie à partir d'images radiographiques.

Cette section explore l'évolution de l'architecture Inception, ses principes fondamentaux et les bénéfices apportés à chaque version jusqu'à V4, avec un focus particulier sur Inception V3.

2.2 Contexte et Problématique

L'évolution des CNNs repose historiquement sur l'augmentation de leur complexité architecturale, notamment à travers l'ajout de couches supplémentaires ou l'élargissement des dimensions des filtres. Si cette démarche permet théoriquement d'améliorer les performances, elle entraîne cependant plusieurs limites majeures :

- **Explosion du nombre de paramètres :** L'accroissement du nombre de couches ou de neurones augmente exponentiellement le nombre de poids à apprendre, ce qui rend l'entraînement plus long, plus coûteux en ressources, et augmente le risque de surapprentissage (overfitting).
- **Coût computationnel élevé :** Les architectures profondes nécessitent d'importantes ressources matérielles pour traiter de grandes quantités de données, ce qui rend leur usage difficile, voire impossible, dans des environnements contraints.
- **Problèmes d'optimisation :** Les réseaux très profonds sont confrontés à des difficultés lors de l'entraînement, comme la disparition ou l'explosion du gradient, qui ralentissent voire empêchent la convergence vers une solution optimale.

L'architecture Inception a été conçue comme une réponse à ces défis. Elle introduit des modules multi-échelles combinant plusieurs tailles de filtres en parallèle, ainsi que des techniques de réduction de dimensionnalité (via des convolutions 1x1, par exemple) qui permettent de limiter le nombre de paramètres tout en préservant la richesse des représentations apprises. Cette approche modulaire et optimisée permet ainsi de concevoir des réseaux à la fois profonds, performants et économes en ressources.

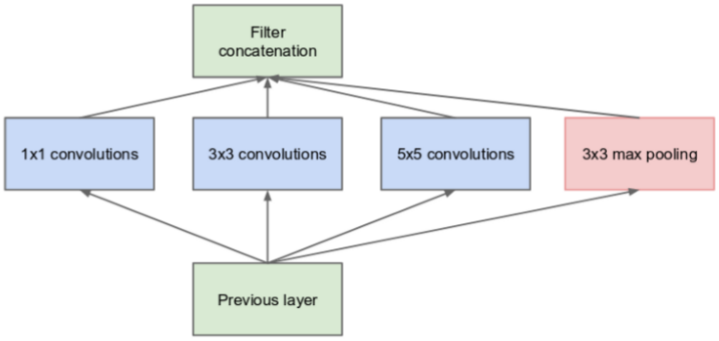
2.3 Principes Fondamentaux du Module Inception

Le cœur de l'architecture Inception réside dans le module Inception, une structure novatrice qui révolutionne la conception des réseaux de neurones convolutifs (CNNs) en effectuant plusieurs opérations convolutives en parallèle.
Contrairement aux CNNs traditionnels, qui appliquent une seule opération à chaque couche (par exemple, une convolution 3×3 ou un max-pooling), le module Inception traite l'entrée simultanément à travers plusieurs branches indépendantes, chacune capturant des caractéristiques à des échelles spatiales distinctes. Ce parallélisme permet au modèle de saisir une diversité de motifs – des détails locaux aux structures globales – au sein d'une même couche, enrichissant ainsi la représentation des données.

Le terme « en parallèle » signifie que les différentes branches du module Inception opèrent indépendamment sur la même carte de caractéristiques d'entrée, sans dépendance séquentielle entre elles. Chaque branche applique une opération spécifique :

- **Convolutions 1×1** : Ces convolutions réduisent la dimensionnalité des cartes de caractéristiques (feature maps) en diminuant le nombre de canaux, ce qui réduit le coût computationnel tout en introduisant des non-linéarités via l'activation ReLU.
- **Convolutions 3×3 et 5×5** :Ces convolutions extraient des motifs à des échelles spatiales variées, capturant des détails fins (3×3) et des structures plus globales (5×5).
- **Max-pooling** : Cette opération conserve les caractéristiques saillantes tout en réduisant la résolution spatiale.

Les sorties de ces branches sont concaténées le long de la dimension des canaux, formant une représentation multi-échelle riche. Cette approche contraste avec les CNNs traditionnels, qui appliquent une seule opération par couche, limitant leur capacité à capturer des motifs à différentes granularités.
La figure ci-dessous illustre la structure du module Inception dans sa version initiale, telle que proposée dans GoogLeNet (Szegedy et al., 2014), avant l'introduction des convolutions 1×1 comme couches de goulot d'étranglement. Cette représentation met en évidence les opérations convolutives et de pooling effectuées en parallèle, sans la réduction de dimensionnalité ultérieurement adoptée.



Limites du Module Inception Initial

Malgré ses avantages, la conception initiale du module Inception présentait des inconvénients :

- **Coût computationnel des convolutions 3×3 et 5×5** : Ces opérations, appliquées directement sur des cartes de caractéristiques à haute dimensionnalité, généraient un grand nombre de paramètres, augmentant le coût d'entraînement.
- **Complexité d'entraînement** : Les ressources limitées rendaient difficile l'entraînement efficace de ces modèles sur de grands ensembles de données.

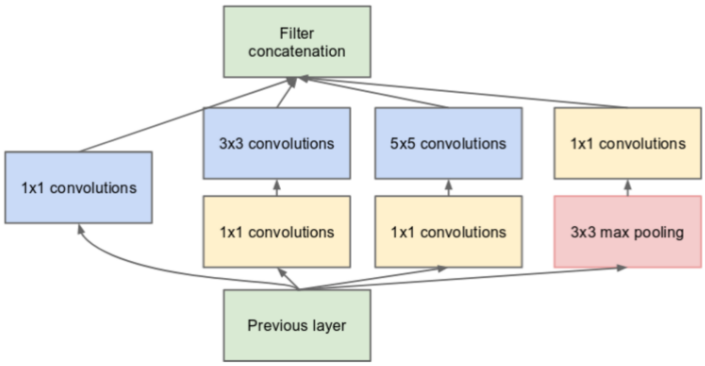
Afin de remédier à ces limitations, Szegedy et ses collaborateurs ont introduit, dès la version Inception v2 en 2015, l'utilisation des convolutions 1×1 comme couches de goulot d'étranglement (**bottleneck layers**).

Convolutions 1×1 comme Goulot d'Étranglement

Les convolutions 1×1 opèrent sur chaque pixel indépendamment, en modifiant la profondeur des cartes de caractéristiques sans altérer leurs dimensions spatiales. Leur utilisation comme couches de goulot d'étranglement offre plusieurs avantages :

- **Réduction de la dimensionnalité** : En diminuant le nombre de canaux avant d'appliquer des convolutions plus coûteuses (3×3 ou 5×5), elles réduisent drastiquement le nombre de paramètres.
- **Diminution du coût computationnel** : Moins de canaux implique moins d'opérations de multiplication, accélérant les calculs.
- **Augmentation de la capacité de modélisation** : Les convolutions 1×1 introduisent des non-linéarités supplémentaires via ReLU, enrichissant la puissance expressive du réseau.

Par exemple, considérons une convolution 5×5 appliquée à une entrée de 256 canaux, nécessitant environ 6400 poids par pixel. En réduisant d'abord les canaux à 64 via une convolution 1×1, le coût tombe à environ 1 600 poids, soit une réduction de 75 %. Cette stratégie, appelée bottleneck, compresse les informations tout en préservant leur contenu essentiel.



La figure ci-dessus illustre le module Inception optimisé, intégrant des convolutions 1×1 comme goulots d'étranglement pour réduire la dimensionnalité avant les convolutions 3×3 et 5×5.

2.4 Évolution de l'Architecture : De Inception V1 à V3

L'architecture Inception a évolué à travers plusieurs versions, chacune apportant des améliorations significatives :

— **Inception V1** ***(GoogLeNet, 2014)*** :
Première itération de l'architecture, Inception V1 a introduit le concept novateur de modules Inception — des blocs multi-branches combinant différentes tailles de filtres (1×1, 3×3, 5×5) et des couches de pooling. Cette structure permettait d'extraire des caractéristiques à plusieurs échelles simultanément, tout en maintenant un nombre de paramètres raisonnable. Grâce à cette approche, GoogLeNet a remporté le défi ILSVRC 2014, surpassant les performances de modèles comme VGG avec une profondeur de 22 couches et un coût computationnel fortement réduit.

— **Inception V2** ***(2015)*** :
Cette version a apporté des améliorations clés pour réduire la charge computationnelle et stabiliser l'entraînement. Elle a introduit la factorisation des convolutions, notamment en remplaçant une convolution coûteuse de 5×5 par deux convolutions consécutives de 3×3, tout en maintenant une zone réceptive équivalente. Elle a également généralisé l'usage des convolutions 1×1 comme couches de réduction de dimensionnalité (bottleneck layers), ce qui permettait d'alléger le modèle avant l'application de convolutions plus lourdes. Ces choix ont permis d'approfondir le réseau tout en assurant une meilleure stabilité numérique.

— **Inception V3** ***(2015)*** :
Version avancée, intégrant des optimisations supplémentaires, telles que la factorisation asymétrique, la normalisation par lots (batch normalization), et des techniques de régularisation comme le label smoothing.
C'est également la version utilisée dans notre projet de détection de pneumonie à partir d'images radiographiques, en raison de son excellent compromis entre précision, stabilité et coût computationnel raisonnable.

Ces améliorations seront détaillées dans la section suivante.

— **Inception V4** ***(2016)*** :
Version plus profonde et optimisée, combinant des blocs Inception enrichis avec des techniques de régularisation avancées. Elle offre une performance encore améliorée, mais au prix d'une complexité et d'un besoin en ressources plus élevés. Inception V4 prépare aussi l'intégration naturelle avec les connexions résiduelles, donnant naissance à l'architecture Inception-ResNet.

2.5 Architecture et Innovations d'Inception V3

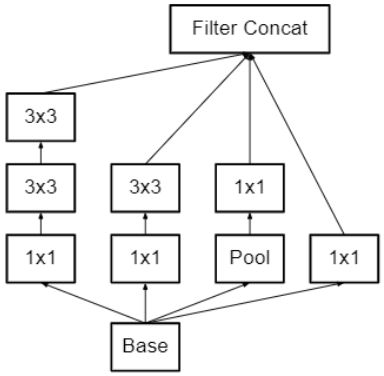
Inception V3 représente une avancée majeure dans l'optimisation des CNNs, combinant efficacité computationnelle et précision accrue. Ses innovations clés incluent :

2.5.1 Factorisation des Convolutions

La factorisation des convolutions réduit la complexité des opérations tout en préservant la capacité de représentation. Par exemple :

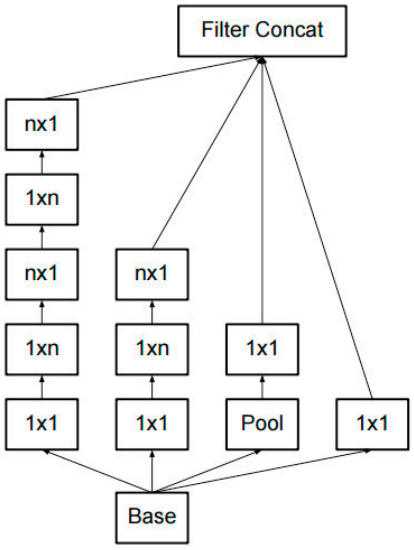
— Une convolution 5×5 (25 paramètres par filtre) est remplacée par deux convolutions 3×3 consécutives (18 paramètres au total), couvrant une zone réceptive similaire.

Cette factorisation permet d'augmenter la profondeur du réseau sans alourdir le coût computationnel.



2.5.2 Factorisation Asymétrique

Inception V3 pousse la factorisation encore plus loin en adoptant des convolutions asymétriques. Par exemple, une convolution n×n (comme 3×3 ou 5×5) peut être factorisée en une séquence de convolutions 1×n suivie de n×1. Cette transformation permet de réduire significativement le nombre de paramètres et les calculs, tout en conservant une zone réceptive équivalente. Elle augmente également la profondeur du réseau, ce qui améliore la capacité de généralisation du modèle et accélère l'entraînement.
L'image ci-dessous illustre cette approche de factorisation, qui constitue l'une des innovations majeures introduites dans Inception V3.



2.5.3 Normalisation par Lots (Batch Normalization)

La normalisation par lots, introduite par Ioffe et Szegedy (2015), est une technique clé pour améliorer la stabilité et l'efficacité de l'entraînement des réseaux de neurones convolutifs profonds, comme Inception V3. Elle réduit l'**internal covariate shift**, l'instabilité des distributions des activations due aux mises à jour des poids, accélérant la convergence et permettant des taux d'apprentissage plus élevés.
Dans Inception V3, elle est appliquée après chaque couche convolutive.

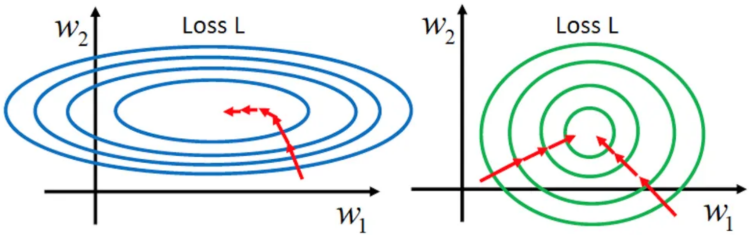
Principe

La normalisation par lots normalise les activations x_i d'une couche pour un mini-lot de taille m :

1. **Moyenne du mini-lot** : $\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i$
2. **Variance du mini-lot** : $\sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2$
3. **Normalisation** : $\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$, où ϵ (ex. 10^{-5}) évite la division par zéro.
4. **Mise à l'échelle et décalage** : $y_i = \gamma \hat{x}_i + \beta$, avec γ et β apprenables.

En inférence, des moyennes et variances globales (calculées sur l'ensemble d'entraînement) sont utilisées pour des prédictions cohérentes.

La normalisation par lot (Batch Normalization) présente plusieurs avantages. Elle permet tout d'abord une stabilisation de l'apprentissage en réduisant le phénomène de internal covariate shift, ce qui facilite la mise à jour stable des poids. Elle contribue également à une convergence plus rapide, en autorisant des taux d'apprentissage plus élevés et en réduisant le nombre d'itérations nécessaires.

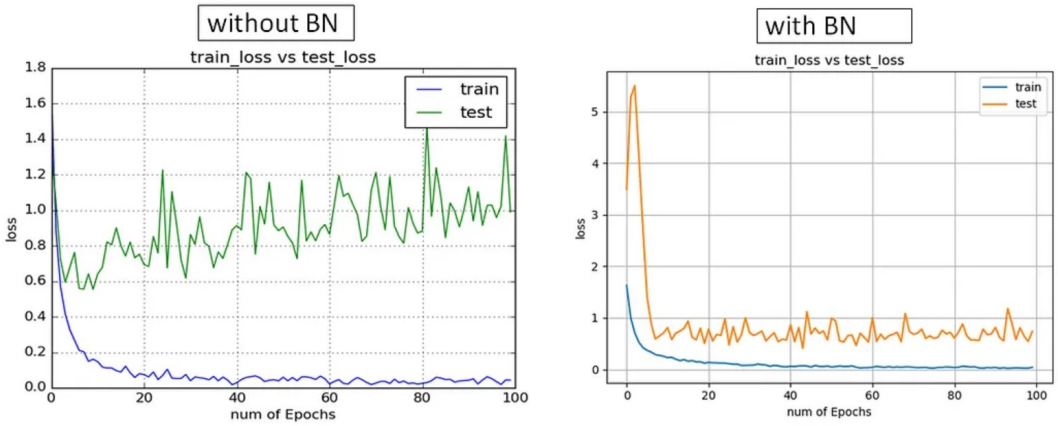


Effet de la Batch Normalization sur les distributions internes des activations dans un réseau de neurones (source : Medium).

De plus, elle renforce la robustesse du modèle en le rendant moins sensible au choix des hyperparamètres. Enfin, elle joue un rôle de régularisation, en introduisant un léger bruit aléatoire durant l'apprentissage, ce qui aide à prévenir le surapprentissage.

Malgré ses nombreux atouts, la normalisation par lot présente quelques limites. Elle dépend de la taille du mini-lot, car des lots trop petits peuvent produire des estimations instables des statistiques ; ce problème peut toutefois être atténué en utilisant des tailles de mini-lots comprises entre 32 et 64. Elle entraîne aussi un léger surcoût computationnel, bien que celui-ci soit généralement compensé par la rapidité de convergence qu'elle offre. Enfin, lors de la phase d'inférence, elle requiert l'utilisation de statistiques globales précalculées, ce qui peut compliquer légèrement le déploiement.

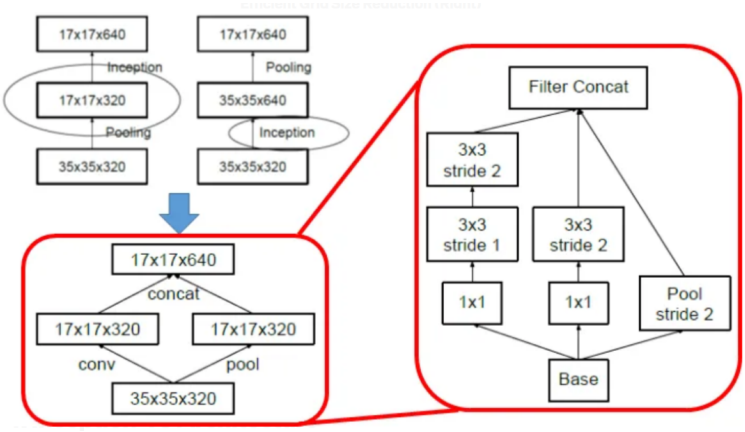
La figure suivante compare les courbes d'apprentissage avec et sans normalisation, illustrant son impact sur la convergence.



2.5.4 Réduction Intelligente des Cartes de Caractéristiques

Dans les réseaux de neurones convolutifs (CNNs), réduire les dimensions spatiales des cartes de caractéristiques (feature maps) est essentiel pour limiter le coût computationnel et extraire des représentations abstraites. Traditionnellement, le max-pooling (par exemple, 2×2 avec pas=2) est utilisé, mais il entraîne une perte d'information en ne conservant que les valeurs maximales, ce qui peut dégrader les détails fins. Inception V3 introduit une réduction intelligente en utilisant des convolutions avec un pas (stride) supérieur à 1, qui préservent davantage d'informations par rapport au max-pooling traditionnel.

Une convolution avec pas (stride>1) réduit la taille des cartes en combinant les activations via un filtre appris, contrairement au max-pooling qui sélectionne uniquement les maximums. Par exemple, une convolution 3×3 avec stride=2 transforme une carte 28×28 en 14×14, capturant des motifs complexes grâce aux poids optimisés. Cette méthode améliore la qualité des caractéristiques extraites dans les couches profondes.



Conventional downsizing (Top Left), Efficient Grid Size Reduction (Bottom Left), Detailed Architecture of Efficient Grid Size Reduction (Right)

L'image ci-dessous illustre clairement la différence entre le max-pooling traditionnel et la réduction par convolution avec pas (stride). Le max-pooling 2×2 avec stride=2 sélectionne simplement la valeur maximale dans chaque bloc, réduisant ainsi la taille de la carte tout en perdant une partie significative de l'information. En revanche, la convolution 3×3 avec stride=2 applique un filtre appris sur des blocs de l'image, ce qui permet non seulement de réduire les dimensions spatiales, mais aussi d'extraire des motifs complexes grâce à l'optimisation des poids.

2.5.5 Classificateurs Auxiliaires et Régularisation

L'entraînement des réseaux de neurones très profonds, comme Inception V3, pose un défi majeur : la disparition du gradient (vanishing gradient). Ce phénomène se manifeste lorsque, au fur et à mesure de la rétropropagation, les gradients deviennent de plus en plus faibles, notamment dans les premières couches du réseau. Résultat : ces couches apprennent très lentement, voire pas du tout, ce qui freine la convergence globale du modèle.

Pour pallier ce problème, Inception V3 introduit des classificateurs auxiliaires (auxiliary classifiers). Il s'agit de petits sous-réseaux insérés à mi-parcours du réseau principal, généralement après certains modules Inception intermédiaires. Leur objectif est double :

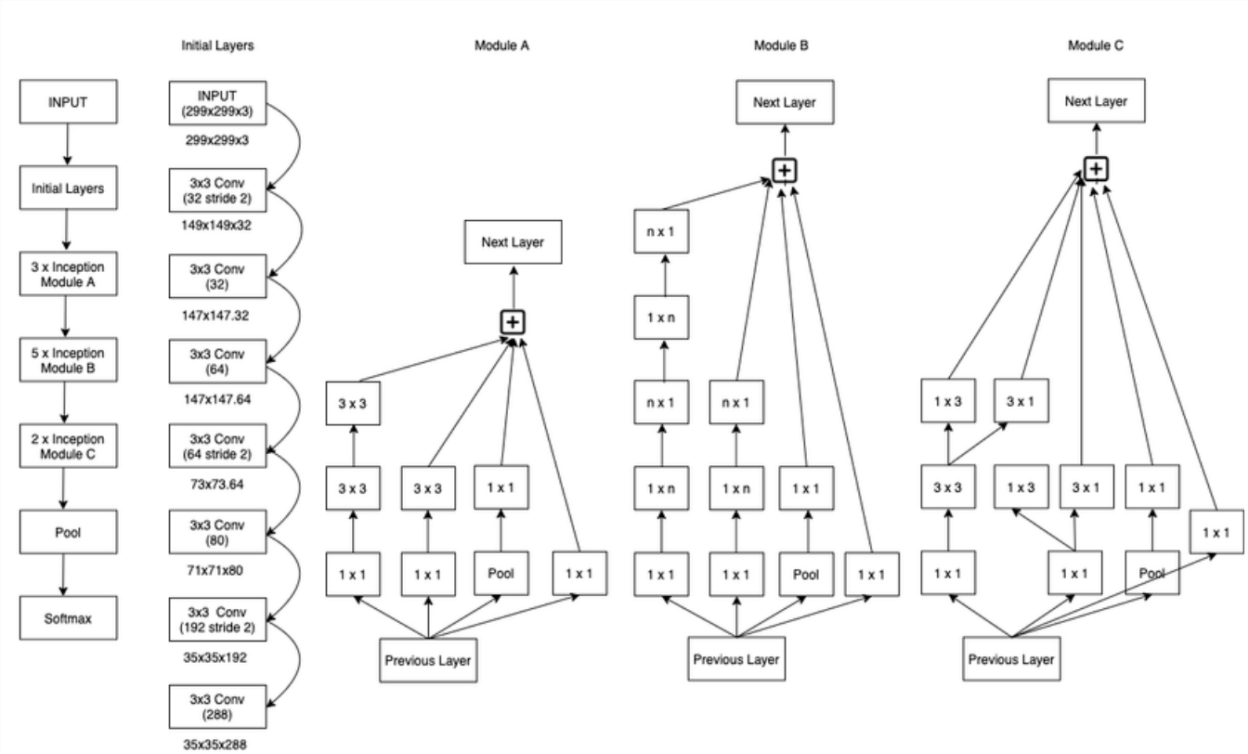
— D'une part, renforcer le signal d'apprentissage dans les couches initiales en injectant un objectif supplémentaire lors de l'entraînement.
— D'autre part, agir comme forme de régularisation, réduisant le risque que le réseau se spécialise trop sur les couches finales.

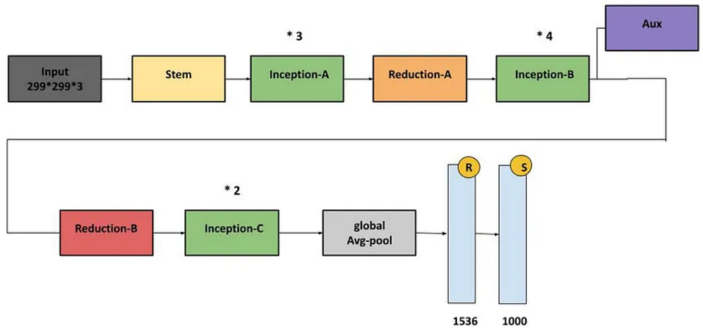
Chaque classificateur auxiliaire se compose généralement d'une couche de pooling, suivie d'un ou plusieurs couches fully-connected, puis d'une couche de classification avec une fonction softmax. La prédiction générée est comparée à la vérité terrain via une fonction de perte (souvent l'entropie croisée), et cette perte est combinée à la perte principale avec un coefficient de pondération réduit (par exemple : 0.3). Ce mécanisme encourage une meilleure propagation du gradient vers les premières couches, stabilisant ainsi l'apprentissage du réseau.

Par ailleurs, Inception V3 adopte une technique de régularisation appelée label smoothing. Dans un cadre classique, les étiquettes des classes sont représentées sous forme de vecteurs one-hot : la classe correcte est codée par un 1, toutes les autres par des 0. Cette rigidité pousse le modèle à devenir extrêmement confiant dans ses prédictions, ce qui peut le rendre sensible au bruit et au surapprentissage.

Le label smoothing atténue cette extrémité en répartissant légèrement la probabilité sur les classes non correctes. Par exemple, au lieu de [1, 0, 0], une sortie cible pourrait devenir [0.9, 0.05, 0.05]. Cela empêche le modèle d'être excessivement sûr de lui, le pousse à généraliser davantage, et améliore la robustesse face à des données bruitées ou ambiguës.

2.6 Architecture Inception : Structure et Organisation des Modules Convolutifs





La figure ci-dessus illustre l'architecture globale d'Inception V3. Elle se compose principalement de trois grandes parties : les couches initiales de convolution, les modules Inception (A, B et C) répartis en plusieurs blocs, et enfin une tête de classification. Les modules Inception, disposés en séquences, permettent d'extraire des caractéristiques à différentes échelles, tandis que les transitions entre les blocs assurent une réduction progressive de la dimensionnalité. Cette organisation hiérarchique favorise une extraction riche et efficace des représentations visuelles en profondeur.

2.6.1 Couches initiales

Le traitement commence par une image RGB de dimensions 299×299×3. Cette entrée traverse une série de convolutions classiques, dont certaines avec des strides de 2, permettant à la fois une réduction progressive de la résolution spatiale et une augmentation de la profondeur (nombre de filtres).
Par exemple :

Une convolution 3×3 avec 32 filtres et stride 2 transforme l'image en 149×149×32.

D'autres convolutions similaires (3×3, puis 64 filtres, etc.) s'enchaînent jusqu'à atteindre des dimensions de 35×35×288 avant l'entrée dans les modules Inception.

Ces couches initiales préparent le tenseur d'entrée en extrayant des caractéristiques de bas niveau et en réduisant les dimensions pour limiter le coût de calcul dans les couches profondes.

2.6.2 Modules Inception : principes et variantes

a. Principe fondamental (Rappel)
L'idée des modules Inception est d'exécuter plusieurs transformations en parallèle sur la même entrée : convolutions de tailles différentes, pooling, etc. Ensuite, les sorties sont concaténées sur le plan des canaux (profondeur). Cela permet au réseau d'apprendre à la fois des motifs fins (avec des petites convolutions), des structures larges (avec des grandes convolutions ou pooling), et des réductions de dimension efficaces.

b. Inception Module A
Le module A repose sur des blocs simples :

- Une branche applique une convolution 1×1 (réduction de dimension).
- Une autre applique une convolution 1×1 suivie d'une 3×3.
- Une autre encore applique une séquence 1×1 → 3×3 → 3×3, qui équivaut à une 5×5 plus efficace.
- Enfin, une branche applique du max-pooling suivi d'une 1×1.

Toutes ces branches travaillent en parallèle, et leurs sorties sont concaténées. Ce module vise à capturer les informations à différentes échelles tout en contrôlant la croissance du nombre de paramètres via les convolutions 1×1.

c. Inception Module B – Factorisation asymétrique
Ce module introduit une optimisation clé : la factorisation des grandes convolutions. Une convolution 7×7, par exemple, est coûteuse. Au lieu de cela, le module B la remplace par deux convolutions : 1×7 suivie de 7×1. Cela réduit :

- Le nombre de paramètres (car 1×n + n×1 < n×n),
- La complexité (meilleure vectorisation),
- Introduit plus de non-linéarités entre les couches, ce qui peut aider à l'apprentissage.

En résumé, ce module permet d'approfondir le réseau sans en exploser la taille ni le coût de calcul.

d. Inception Module C – Factorisation plus fine
Le module C raffine encore cette logique. Il utilise des combinaisons 1×3 et 3×1 à la place des 3×3 classiques. Ces blocs permettent une plus grande profondeur effective du réseau avec un coût calculatoire réduit.
Par exemple :

- Une branche effectue 1×1 → 1×3 → 3×1.
- Une autre applique du pooling suivi d'une convolution 1×1.
- Toutes les branches se terminent par des concaténations de sorties.

Ce module est particulièrement utile dans les couches profondes, où le réseau a besoin de diversité de représentation sans perte de performance.

2.6.3 Fusion des branches

Dans chaque module, après l'exécution parallèle des transformations, les sorties sont concaténées sur la dimension des canaux. Cela donne au réseau une représentation riche, intégrant des informations locales et globales, fines et larges, linéaires et non-linéaires.

2.6.4 Étapes finales

Après les modules Inception, le réseau passe par :

- Quelques couches convolutives supplémentaires (par exemple, un 3×3 avec 192 filtres et stride 2),
- Une couche de pooling global pour réduire la dimension spatiale à 1×1,
- Une couche Softmax qui attribue une probabilité à chaque classe de sortie.

2.7 Avantages et Limites d'Inception V3

Inception V3 offre plusieurs avantages :

- **Efficacité computationnelle :** Grâce aux convolutions 1×1 et à la factorisation, le modèle atteint une précision élevée avec un coût réduit par rapport à des architectures comme VGG ou AlexNet.
- **Flexibilité :** Les modules Inception capturent des caractéristiques à différentes échelles, rendant le modèle polyvalent pour diverses tâches de vision par ordinateur.
- **Évolutivité :** L'architecture peut être combinée avec d'autres approches, comme les connexions résiduelles (Inception-ResNet), pour de meilleures performances.

Cependant, Inception V3 présente des limites :

- **Complexité architecturale :** La structure multi-branches rend le modèle plus difficile à concevoir et à déboguer par rapport à des architectures plus simples.
- **Dépendance aux ressources :** Bien qu'optimisé, l'entraînement sur de grands ensembles de données nécessite encore des GPU puissants.

2.8 Conclusion

L'architecture Inception, et en particulier Inception V3, illustre une approche novatrice pour concevoir des CNNs profonds et efficaces. En combinant des modules multi-échelles, des convolutions factorisées, et des techniques d'optimisation comme la normalisation par lots, Inception V3 atteint un équilibre remarquable entre précision et efficacité computationnelle. Dans le cadre de notre projet de détection de la pneumonie, ce modèle a démontré sa robustesse et sa capacité à généraliser à partir de données médicales complexes. Ces avancées font d'Inception V3 un outil puissant pour les applications de vision par ordinateur, avec un potentiel d'extension à d'autres domaines de l'apprentissage profond.

3 Modèle ResNet

3.1 Présentation de ResNet et motivation

Avec l'essor des réseaux de neurones profonds dans le domaine de la vision par ordinateur, une tendance s'est rapidement imposée : augmenter la profondeur des modèles pour améliorer les performances. Des architectures comme VGG ou Inception ont ainsi démontré que des réseaux plus profonds pouvaient capturer des caractéristiques plus complexes et plus abstraites, ce qui conduit à de meilleurs résultats sur des tâches telles que la classification d'images.

Cependant, cette augmentation de profondeur a révélé un problème majeur : au-delà d'un certain point, ajouter des couches supplémentaires dégrade les performances, au lieu de les améliorer. Ce phénomène contre-intuitif, appelé **dégradation du gradient**, rend l'entraînement des réseaux très profonds difficile. En effet, même avec des techniques comme la normalisation de batch ou l'optimisation avancée, le réseau a tendance à **stagner ou régresser**, les gradients devenant trop faibles pour mettre à jour correctement les poids.

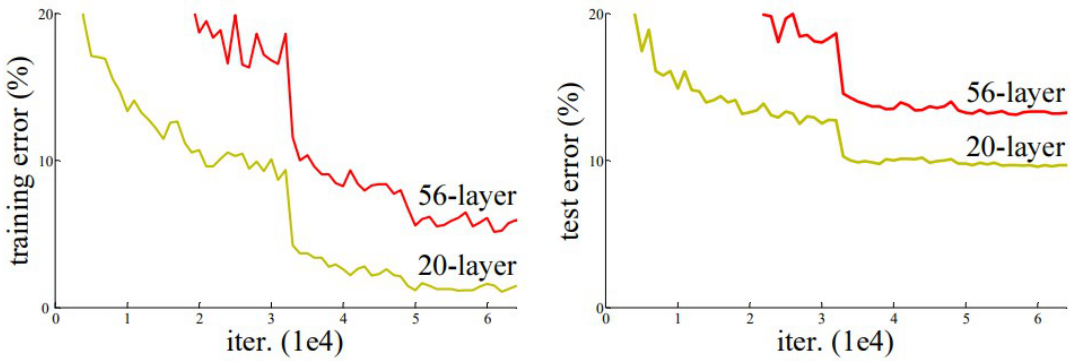


Figure : Comparison of 20-layer vs 56-layer architecture.

Ici dans ce graphe, on peut observer qu'un réseau de neurones convolutifs (CNN) de 56 couches présente un taux d'erreur plus élevé sur les ensembles de données d'entraînement et de test qu'une architecture de 20 couches. Après une analyse plus approfondie du taux d'erreur, les auteurs ont pu conclure que cela était causé par le problème de disparition/explosion du gradient.

C'est dans ce contexte qu'est née l'architecture **ResNet (Residual Network)**, proposée par **He et al.** en 2015. L'idée centrale de ResNet est simple mais puissante : permettre aux couches de **sauter** certaines transformations en ajoutant des **connexions résiduelles**. Ces connexions permettent au réseau d'apprendre non pas une transformation directe des données, mais la différence (ou « résidu ») entre l'entrée et la sortie attendue.

Grâce à cette approche, ResNet a pu s'entraîner efficacement même avec **plus de 100 couches**, sans subir les problèmes de dégradation qui freinaient les architectures précédentes. Cette innovation a marqué un tournant majeur dans le développement des réseaux de neurones profonds et a permis d'atteindre des performances de pointe dans des compétitions telles qu'ImageNet.

3.2 Architecture de ResNet-50

L'architecture ResNet-50 est une version profonde de la famille des réseaux résiduels. Comme son nom l'indique, elle est composée de **50 couches** comportant des poids entraînables. Elle repose sur deux éléments essentiels : les **blocs résiduels** et les **couches de type "bottleneck"**.

Blocs résiduels

Le principe fondamental de ResNet repose sur l'utilisation de blocs dits **résiduels**, qui permettent d'ajouter l'entrée initiale du bloc à sa sortie transformée. Formellement, au lieu d'apprendre une fonction de mapping $H(x)$, le réseau apprend une fonction résiduelle $F(x) = H(x) - x$, puis reconstitue l'output via $H(x) = F(x) + x$. Cela facilite l'apprentissage et évite le problème de la dégradation observé dans les architectures classiques.

Chaque bloc résiduel inclut donc une **connexion directe (skip connection)** qui contourne certaines couches.

Structure "Bottleneck"

Pour construire une architecture aussi profonde sans explosion du coût computationnel, ResNet-50 utilise une structure appelée **bottleneck**. Chaque bloc bottleneck contient trois couches convolutives :

- une couche 1×1 pour réduire la dimension,
- une couche 3×3 pour le traitement principal,
- une couche 1×1 pour restaurer la dimension d'origine.

Cette structure permet d'optimiser le nombre de calculs tout en conservant une bonne expressivité.

Composition de l'architecture ResNet-50

L'architecture complète peut être résumée ainsi :

- Une couche de convolution initiale 7×7 avec 64 filtres, stride 2, suivie d'un **max pooling** 3×3 ;
- Un premier bloc contenant 3 unités bottleneck avec 256 canaux de sortie ;
- Un deuxième bloc contenant 4 unités bottleneck avec 512 canaux ;
- Un troisième bloc contenant 6 unités bottleneck avec 1024 canaux ;
- Un quatrième bloc contenant 3 unités bottleneck avec 2048 canaux ;
- Une couche de **global average pooling** ;
- Une couche fully connected avec softmax pour la classification finale.

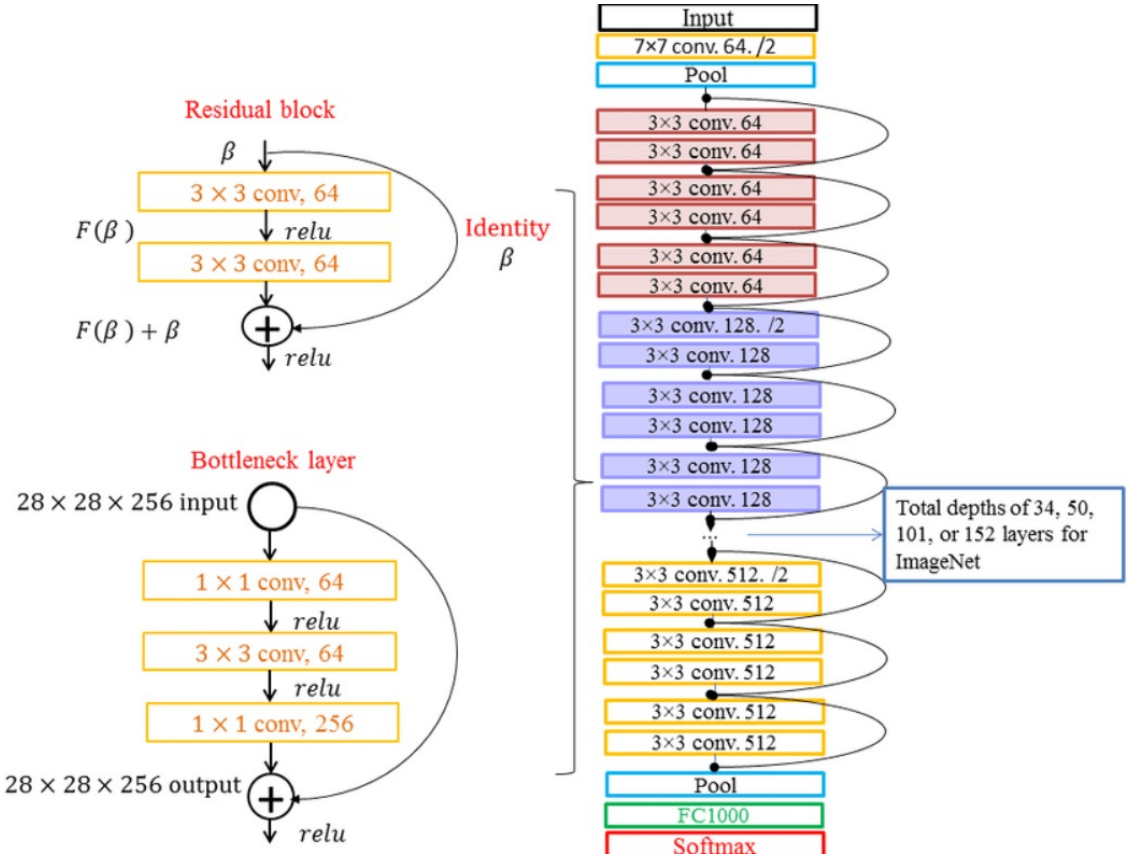


Figure : Schéma de l'architecture ResNet-50 montrant les différentes couches et blocs résiduels.

3.3 Fonctionnement et particularités techniques

Au-delà de sa structure architecturale, ResNet-50 repose sur un ensemble de choix techniques qui optimisent son fonctionnement et facilitent son entraînement, malgré sa grande profondeur.

Propagation avant (forward) et blocs résiduels

Dans chaque bloc résiduel, l'entrée est propagée à travers une série de couches convolutives, mais aussi **copiée directement vers la sortie via une skip connection**. Cette opération d'addition élément par élément assure que l'information d'origine est toujours préservée, même si les transformations intermédiaires échouent partiellement à capturer une représentation utile.

Cela permet également d'accélérer la convergence du modèle en facilitant le passage du gradient lors de l'entraînement.

Initialisation des poids

Un problème courant dans les réseaux profonds est l'**explosion ou la disparition des gradients**. Pour y remédier, ResNet-50 bénéficie d'une initialisation soigneusement choisie, notamment :

- l'utilisation d'une initialisation **He** (aussi appelée *Kaiming*), adaptée aux fonctions d'activation ReLU,
- une structuration en blocs courts (bottlenecks) pour limiter les chaînes de transformations trop longues sans régularisation.

Normalisation de batch (BatchNorm)

Chaque couche convolutive est suivie d'une **Batch Normalization**, une technique qui permet de stabiliser l'apprentissage en normalisant les activations à chaque mini-batch. Cela :

- accélère la convergence,
- permet d'utiliser des taux d'apprentissage plus élevés,
- réduit le besoin de techniques de régularisation supplémentaires.

Dans ResNet-50, la normalisation est appliquée **avant** la fonction d'activation ReLU, ce qui permet une meilleure stabilité numérique.

Fonction d'activation

Comme la plupart des architectures modernes, ResNet-50 utilise la fonction d'activation **ReLU (Rectified Linear Unit)**. Cette fonction introduit la non-linéarité tout en conservant une bonne propagation du gradient, contrairement à des fonctions plus anciennes comme *sigmoïde* ou *tanh*.

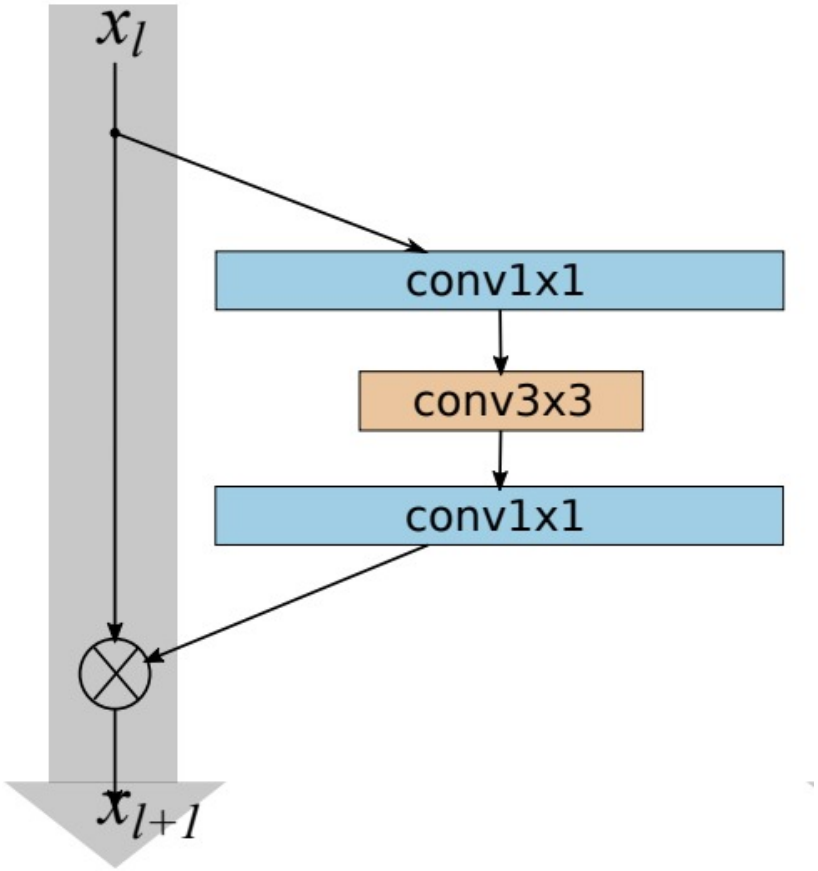
Efficacité computationnelle

Malgré ses 50 couches, ResNet-50 reste relativement efficace en termes de calcul grâce à :

— l'utilisation de couches 1×1, bien moins coûteuses que les couches 3×3,

— la réduction progressive de la taille des cartes de caractéristiques (feature maps) au fil du réseau,

— le partage d'opérations via les connexions résiduelles.



(b) bottleneck

Figure : Illustration d'un bloc bottleneck résiduel avec les couches 1×1, 3×3 *et la connexion directe.*

3.4 Applications et performances

Depuis sa création, ResNet-50 est devenu une architecture de référence dans le domaine de la vision par ordinateur, grâce à son excellent compromis entre profondeur, performance et stabilité d'entraînement.

Performances sur ImageNet

ResNet-50 a été initialement proposé pour le challenge ImageNet Large Scale Visual Recognition Challenge (ILSVRC) 2015. Il y a remporté la première place dans la tâche de classification d'images avec un taux d'erreur **top-5 de seulement 3.6%**, surpassant largement les architectures précédentes comme VGG-16 ou GoogleNet.

Le modèle a démontré sa capacité à généraliser efficacement tout en étant plus léger que certains réseaux équivalents en profondeur sans connexions résiduelles.

Applications concrètes

ResNet-50 est utilisé dans un large éventail d'applications en vision par ordinateur, telles que :

- **Détection d'objets** (avec Faster R-CNN, Mask R-CNN, etc.) où ResNet-50 est souvent utilisé comme backbone,
- **Reconnaissance faciale** grâce à sa capacité à extraire des descripteurs puissants,
- **Segmentation d'images** et **analyse médicale** (IRM, radiologie, etc.) où la robustesse des features est cruciale,
- **Transfert de style** et autres techniques d'apprentissage profond utilisant des features extraits de ResNet.

Sa modularité, son efficacité et sa large adoption dans les bibliothèques comme PyTorch ou TensorFlow en font un modèle standard pour le prototypage comme pour la mise en production.

3.5 Limites de ResNet

Bien que ResNet-50 ait marqué une avancée majeure dans l'entraînement des réseaux très profonds, certaines limitations subsistent, particulièrement lorsque l'on cherche à étendre l'architecture à des versions plus complexes ou à des tâches spécifiques.

Profondeur extrême et efficacité

L'idée des connexions résiduelles permet d'entraîner des réseaux beaucoup plus profonds, comme ResNet-101 ou ResNet-152. Toutefois, ces versions profondes introduisent :

- une **augmentation considérable du temps d'entraînement**,
- une **consommation mémoire importante**, limitant leur utilisation sur des machines à ressources modestes,
- une tendance à **apprendre des représentations redondantes**, surtout lorsque les données sont limitées.

Ainsi, l'ajout de couches ne garantit pas toujours une amélioration significative des performances.

Redondance et surparamétrisation

Plusieurs études ont montré que certaines couches intermédiaires de ResNet peuvent apprendre des fonctions très similaires, ce qui indique une **surparamétrisation**. Cela peut entraîner une complexité inutile du modèle, particulièrement pour les tâches simples ou les jeux de données restreints.

Dépassé par des architectures récentes

Des architectures plus modernes comme :

- **EfficientNet**, qui optimise à la fois la profondeur, la largeur et la résolution de manière équilibrée,
- **MobileNet**, qui vise la légèreté pour les appareils mobiles,
- ou encore les **Vision Transformers (ViT)**, qui abandonnent entièrement les convolutions,

ont surpassé ResNet dans certaines tâches tout en réduisant le nombre de paramètres ou le coût computationnel.

Limitations dans des contextes spécifiques

Enfin, ResNet peut être moins performant sur des tâches où la structure spatiale locale est moins pertinente (ex. : traitement de texte ou certaines données médicales tabulaires) ou lorsque les données sont très déséquilibrées, ce qui peut rendre les connexions résiduelles moins efficaces.

3.6 Évolutions de ResNet

Depuis la publication de ResNet en 2015, de nombreuses variantes ont été proposées afin d'améliorer ses performances, sa flexibilité et son efficacité computationnelle.

Profondeur accrue : ResNet-101 et ResNet-152

Les versions plus profondes de ResNet, comme **ResNet-101** et **ResNet-152**, conservent la même logique de blocs résiduels mais augmentent significativement le nombre de couches :

- ResNet-101 : 101 couches, meilleure précision sur ImageNet au prix d'un coût de calcul plus élevé,
- ResNet-152 : 152 couches, performances optimales mais temps d'inférence allongé.

Ces versions sont utilisées lorsqu'un maximum de précision est requis, notamment dans les systèmes de reconnaissance avancée.

ResNeXt : blocs groupés

ResNeXt est une évolution qui introduit l'idée de **grouped convolutions** : au lieu d'une seule transformation convolutive par bloc, plusieurs transformations parallèles sont appliquées et agrégées.

Cela permet d'augmenter la capacité du réseau sans augmenter proportionnellement le nombre de paramètres. On parle ici de la notion de **cardinalité**, un nouveau facteur de variation ajouté à la profondeur et la largeur.

SE-ResNet : attention de canal

L'architecture **SE-ResNet** (Squeeze-and-Excitation ResNet) ajoute un mécanisme d'attention qui permet au réseau de **pondérer dynamiquement l'importance de chaque canal**. Cela améliore la capacité du modèle à se concentrer sur les informations discriminantes, tout en conservant la structure de ResNet.

ResNet v2 : pre-activation

Une autre évolution notable est **ResNet v2**, qui modifie l'ordre des opérations dans les blocs :

- La **Batch Normalization** et la fonction ReLU sont placées **avant** la convolution (pre-activation),
- Ce changement améliore la propagation du gradient et permet un entraînement plus stable dans des réseaux très profonds.

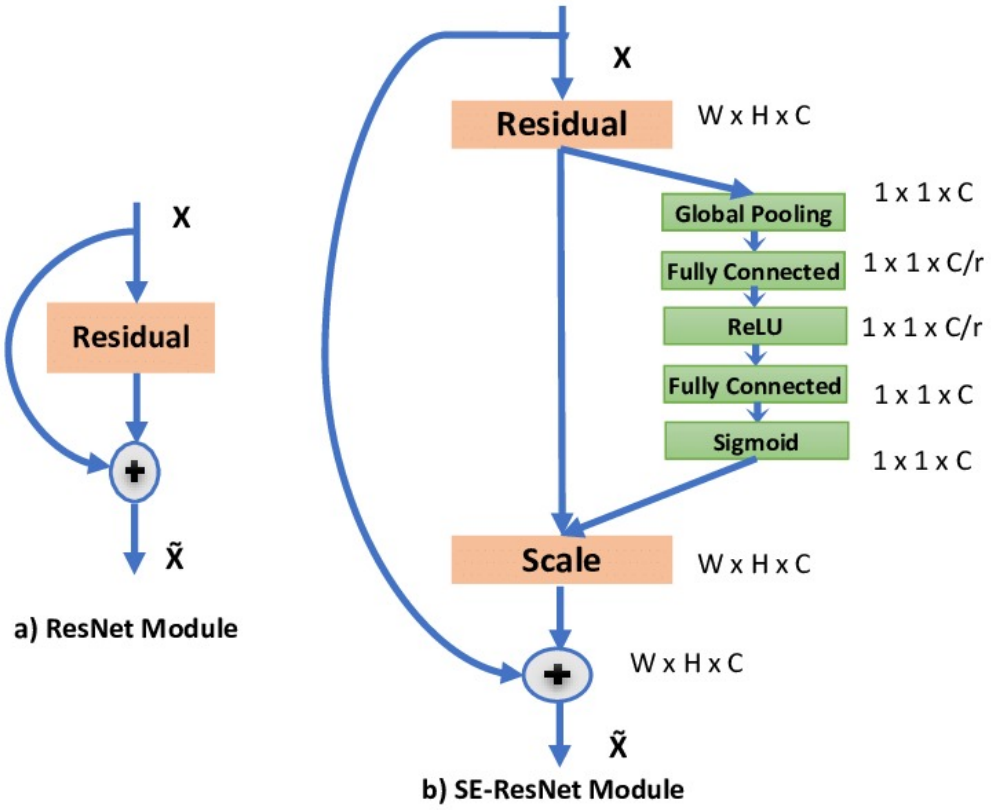


Figure : Comparaison de différents types de blocs résiduels dans ResNet et SE-ResNet.

Impact et adoption

Ces variantes ont été adoptées dans de nombreuses applications industrielles et académiques :

- ResNeXt est utilisé dans des tâches à grande échelle comme la reconnaissance vidéo (e.g. Facebook),
- SE-ResNet est populaire dans les compétitions de vision (ImageNet, Kaggle),
- ResNet v2 est préféré dans les configurations où la stabilité d'entraînement est essentielle.

Ces évolutions témoignent de la robustesse de l'idée initiale de ResNet, qui continue d'influencer la conception des architectures modernes de deep learning.

3.7 Conclusion

L'architecture ResNet-50 représente une avancée majeure dans le domaine de l'apprentissage profond, notamment grâce à son introduction des connexions résiduelles qui ont permis de surmonter le problème de la dégradation des performances dans les réseaux très profonds.

Sa conception équilibrée entre efficacité, profondeur et stabilité en a fait un modèle de référence pour de nombreuses tâches en vision par ordinateur. Bien que certaines limites existent, notamment en termes de coût computationnel et de redondance, les évolutions successives comme ResNeXt, SE-ResNet et ResNet v2 ont permis d'adapter et d'améliorer cette architecture emblématique.

ResNet continue aujourd'hui d'influencer la conception de nouvelles architectures, confirmant son rôle central dans l'histoire récente du deep learning.

4 Modèle DenseNet

4.1 Introduction

Le Deep Learning a transformé l'intelligence artificielle en permettant aux machines d'apprendre à partir de grandes quantités de données non structurées. Cette approche a révolutionné divers domaines, notamment la vision par ordinateur, où les réseaux de neurones profonds peuvent maintenant détecter et classer des objets dans des images avec une précision impressionnante. Dans le contexte médical, l'utilisation de modèles de Deep Learning pour automatiser le diagnostic d'images médicales, telles que les radiographies, offre une aide précieuse aux professionnels de santé pour améliorer la précision et la rapidité des diagnostics.

4.2 Problématique du diagnostic médical automatisé

Dans le domaine médical, le diagnostic repose souvent sur l'interprétation d'images radiologiques. Cependant, l'analyse de ces images est un processus complexe et subjectif qui peut être influencé par des facteurs tels que l'expérience du radiologue et la qualité de l'image. Cela peut entraîner des erreurs de diagnostic, parfois graves. L'automatisation de cette tâche à l'aide de modèles d'intelligence artificielle (IA) pourrait significativement réduire ces erreurs et permettre une détection plus rapide des maladies. Les modèles de Deep Learning, tels que DenseNet101, sont capables d'analyser des images de manière plus rapide et plus précise, offrant ainsi un soutien essentiel aux radiologues.

4.3 Architecture du Modèle DenseNet101

DenseNet-101 est une architecture de réseau de neurones convolutifs profonds qui se distingue par sa capacité à relier chaque couche à toutes les couches précédentes, formant ainsi des connexions denses entre les couches. Cette approche permet de réduire le problème de la disparition des gradients, facilitant ainsi l'entraînement de réseaux très profonds. DenseNet101 comprend 101 couches et utilise un mécanisme de transition pour réduire la dimensionnalité entre les blocs de couches. En raison de sa structure dense, le modèle est particulièrement adapté pour traiter des ensembles de données volumineux et complexes, comme ceux utilisés pour les images médicales.

4.4 Représentation de l'architecture

Le modèle **DenseNet101** est un réseau de neurones convolutifs profond...

Bloc	**Type de couche**	**Dimensions de sortie**
Entrée	Image RGB	$224 \times 224 \times 3$
Initial Convolution	7×7, stride 2 + MaxPool 3×3	$56 \times 56 \times 64$
Dense Block 1	6 couches	$56 \times 56 \times 256$
Transition 1	Conv 1×1 + AvgPool 2×2	$28 \times 28 \times 128$
Dense Block 2	12 couches	$28 \times 28 \times 512$
Transition 2	Conv 1×1 + AvgPool 2×2	$14 \times 14 \times 256$
Dense Block 3	48 couches	$14 \times 14 \times 1024$
Transition 3	Conv 1×1 + AvgPool 2×2	$7 \times 7 \times 512$
Dense Block 4	32 couches	$7 \times 7 \times 1024$
Global Avg Pool	Pooling	$1 \times 1 \times 1024$
FC Layer	Fully Connected	$1 \times 1 \times$ `num_classes`

Voici une représentation linéaire simplifiée de l'architecture du modèle **DenseNet-101**, décrivant chaque étape de traitement d'une image d'entrée jusqu'à la classification finale :

IM Image d'entrée (RGB, taille 224×224×3)
↓
CI Convolution initiale (7×7) + MaxPooling (3×3, stride 2)
↓
BD1 Bloc dense 1 — contient 6 couches
↓
T1 Transition 1 (Convolution 1×1 + Moyenne Pooling 2×2)
↓
BD2 Bloc dense 2 — contient 12 couches
↓
T2 Transition 2 (Convolution 1×1 + Moyenne Pooling 2×2)
↓
BD3 Bloc dense 3 — contient 48 couches
↓
T3 Transition 3 (Convolution 1×1 + Moyenne Pooling 2×2)
↓
BD4 Bloc dense 4 — contient 32 couches
↓
PGM Pooling global moyen (Global Average Pooling)

↓

FC Couche entièrement connectée (Fully Connected)

↓

OUTPUT Sortie (Classification : NORMAL ou PNEUMONIA)

4.5 Préparation des Données et Entraînement

Dans le cadre de ce projet, j'ai travaillé sur un dataset d'images médicales appelé `chest_xray`, la dataset kaggle originale a contenue 16 images pour le dossier "val" ce qui est pas favorable lors de l'utilisation d'un modele comme densNet101 ce qui m'a poussé à effectuer une manipulation qui sert à augmenter la taille de val set à 10 pour cent du train set , donc maitenant la dataset est structuré en trois sous-dossiers : `train` (4813 images), `val` (427 images) et `test` (627 images), répartis entre les classes "NORMAL" et "PNEUMONIA". Ce dataset est utilisé pour entraîner et tester le modèle DenseNet101. Avant d'entraîner le modèle, les images ont été prétraitées, redimensionnées à une taille standard, et normalisées pour s'assurer qu'elles étaient compatibles avec le modèle d'entrée. L'entraînement a été effectué à l'aide d'un ensemble d'entraînement, et les performances du modèle ont été évaluées à l'aide de l'ensemble de validation.

4.6 Limites et inconvénients de DenseNet-101

Malgré ses performances élevées, le modèle **DenseNet-101** présente certaines limites et inconvénients :

- **Complexité computationnelle élevée :** Avec ses 101 couches et ses nombreuses connexions internes, DenseNet-101 nécessite une puissance de calcul importante, ce qui peut être un obstacle pour une utilisation sur des machines aux ressources limitées (comme les appareils mobiles ou les serveurs à faibles capacités).
- **Temps d'entraînement plus long :** En raison de sa profondeur et de sa structure dense, le processus d'entraînement est plus long comparé à des modèles plus légers comme MobileNetV2.
- **Utilisation mémoire :** Les connexions entre toutes les couches augmentent significativement l'utilisation de la mémoire GPU, ce qui peut limiter la taille du batch utilisable pendant l'entraînement.

— **Difficulté de déploiement :** Le déploiement du modèle dans des environnements temps réel (comme les applications embarquées) peut être problématique, notamment à cause de sa taille et de son besoin en ressources.
— **Moins adapté pour les petits jeux de données :** DenseNet-101 est plus performant sur de grands ensembles de données. Sur des jeux de données limités, il peut facilement sur-apprendre (overfitting), sauf si des techniques de régularisation sont bien appliquées.

4.7 Conclusion

Dans ce projet, nous avons exploré l'efficacité du modèle DenseNet101 dans le cadre du diagnostic automatisé à partir d'images radiographiques. Les résultats obtenus montrent que ce modèle offre des performances remarquables, surpassant celles d'autres architectures populaires telles que ResNet50 et MobileNetV2 dans les mêmes conditions expérimentales. Grâce à sa structure dense et à sa capacité à propager efficacement les gradients, DenseNet101 s'est montré particulièrement adapté à la classification d'images médicales.

Ce travail confirme le potentiel des réseaux de neurones profonds dans l'aide au diagnostic médical, en particulier pour des tâches critiques comme la détection de la pneumonie sur les radiographies thoraciques. À l'avenir, des améliorations peuvent être envisagées, telles que l'optimisation des hyperparamètres, l'augmentation des données ou l'intégration de techniques d'ensemble pour encore améliorer la robustesse et la précision du modèle. Cela ouvrirait la voie à une intégration plus large de ces systèmes dans les environnements cliniques.

5 Modèle VGG

5.1 Introduction

La pneumonie est une infection pulmonaire grave qui peut être causée par des bactéries, des virus ou des champignons. Elle affecte les sacs aériens des poumons, provoquant leur inflammation et leur remplissage de liquide, ce qui rend la respiration difficile. Une détection rapide et précise est essentielle pour un traitement efficace, surtout chez les enfants et les personnes âgées, qui sont les plus vulnérables.

Avec l'évolution des technologies d'intelligence artificielle, et notamment du deep learning, de nouvelles méthodes de diagnostic assisté par ordinateur ont vu le jour. Le traitement automatique des images médicales à l'aide de réseaux de neurones profonds permet aujourd'hui de détecter certaines maladies avec une précision remarquable.

Dans le cadre de ce projet, nous avons pour objectif de développer un système de détection automatique de la pneumonie à partir d'images de radiographies thoraciques. Ce travail a été réalisé en groupe, où chaque membre a expérimenté un modèle différent pour la classification binaire (pneumonie / sain). Pour ma part, j'ai choisi d'utiliser VGG16, un modèle de réseau de neurones convolutif pré-entraîné, réputé pour sa robustesse et sa performance en transfert learning.

L'objectif principal de cette partie est donc d'examiner l'efficacité du modèle VGG16 dans la détection de la pneumonie, en analysant sa précision, sa capacité de généralisation, et ses éventuelles limites.

5.2 Présentation du modèle VGG16

5.2.1 Origine du modèle VGG

Les réseaux VGG (Visual Geometry Group) ont été proposés lors de la compétition ILSVRC 2014 (ImageNet Large Scale Visual Recognition Challenge). Ils utilisent des couches convolutionnelles profondes avec de petits filtres de taille 3×3 et des couches de max pooling pour capturer efficacement les hiérarchies spatiales. Les deux principales versions de cette architecture sont :

VGG16 : 16 couches avec des poids apprenables (13 couches convolutionnelles + 3 couches entièrement connectées)

VGG19 : 19 couches avec des poids apprenables (16 couches convolutionnelles + 3 couches entièrement connectées)

Les deux architectures suivent une conception similaire, avec des couches convolutionnelles suivies de max pooling, puis de couches entièrement connectées à la fin.

5.2.2 Structure et architecture du modèle VGG16

Le modèle VGG16 (Visual Geometry Group 16 couches) est un réseau de neurones convolutif profond conçu pour la reconnaissance d'images. Il se distingue par une architecture simple, uniforme et profonde, ce qui en fait un excellent choix pour des tâches de classification, y compris dans le domaine médical.

L'architecture de VGG16 est constituée de 16 couches contenant des poids :

13 couches convolutionnelles,

3 couches entièrement connectées (dense).

Ces couches sont organisées en 5 blocs convolutifs, chacun suivi d'une couche de max pooling pour réduire la taille spatiale des cartes de caractéristiques (feature maps). Chaque couche de convolution utilise des filtres de taille 3x3 avec un pas (stride) de 1 et une fonction d'activation ReLU (Rectified Linear Unit). Cette taille de filtre a été choisie pour capturer de petites caractéristiques tout en gardant un nombre de paramètres modéré.

Voici un résumé du flux dans VGG16 :

Bloc 1 : 2 couches convolutionnelles (64 filtres) + max pooling

Bloc 2 : 2 couches convolutionnelles (128 filtres) + max pooling

Bloc 3 : 3 couches convolutionnelles (256 filtres) + max pooling

Bloc 4 : 3 couches convolutionnelles (512 filtres) + max pooling

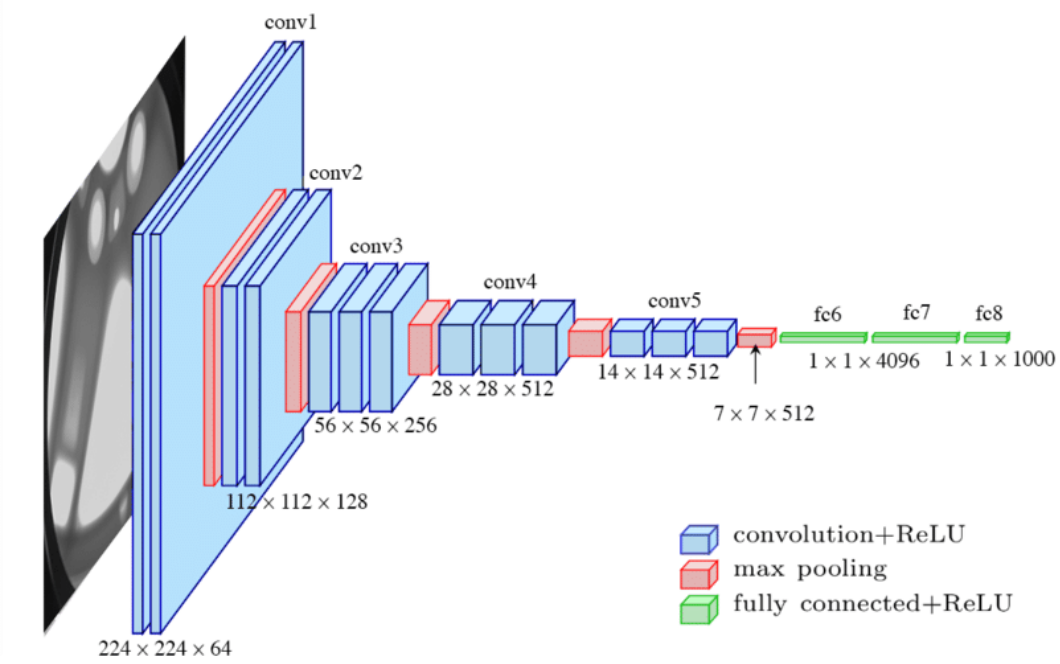
Bloc 5 : 3 couches convolutionnelles (512 filtres) + max pooling

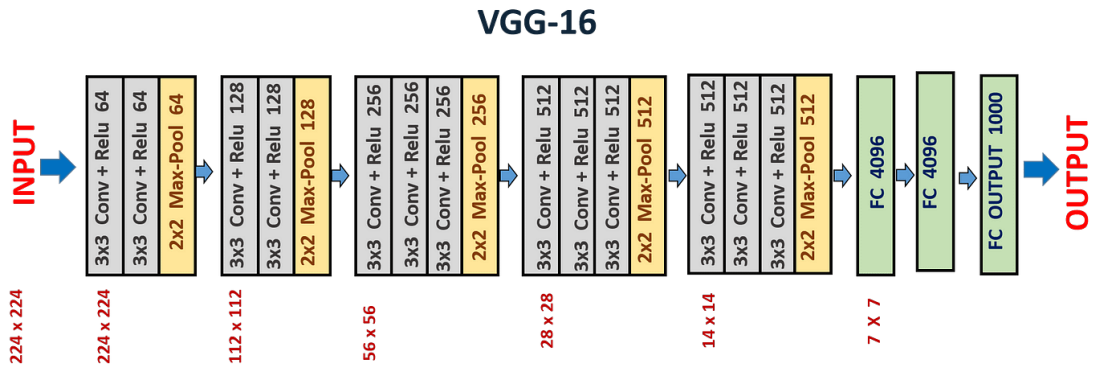
Après ces blocs, les cartes de caractéristiques sont aplaties (flatten), puis passent à travers deux couches fully connected de 4096 neurones, suivies d'une dernière couche fully connected qui contient 1000 neurones dans la version originale (pour les 1000 classes d'ImageNet), mais qui peut être remplacée par une couche

adaptée à notre tâche (par exemple : une couche dense de 1 neurone avec activation sigmoïde pour une classification binaire : pneumonie / sain).

L'entrée du modèle doit être une image de 224x224 pixels avec 3 canaux (RVB). C'est pourquoi les images médicales (radiographies) doivent être redimensionnées et converties correctement avant d'être utilisées en entrée.

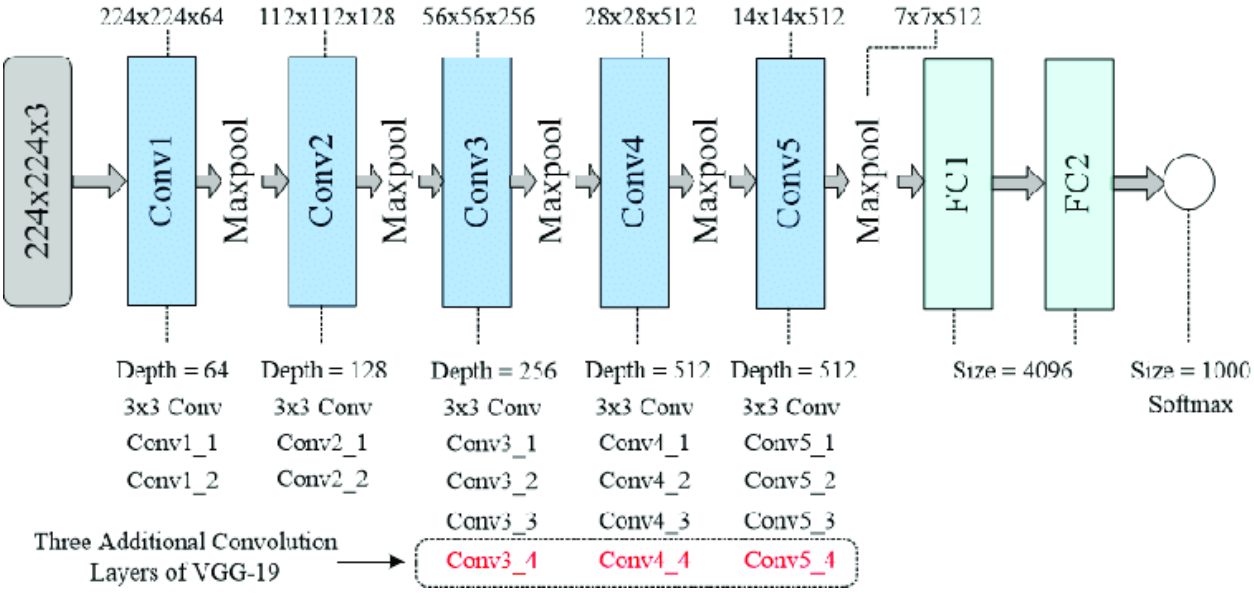
Enfin, lors de l'usage en transfert learning, les poids pré-entraînés de VGG16 (sur ImageNet) sont utilisés pour initialiser les couches convolutionnelles, et seules les dernières couches (couches denses) sont réentraînées sur le jeu de données médical.





5.2.3 Comparaison entre les architectures VGG16 et VGG19

Layer	VGG16	VGG19
Size of Layer	41	47
Image Input Size	224x224 pixel	224x224 pixel
Convolutional Layer	13	16
Filter Size	64 & 128	64,128,256, & 512
ReLU	5	18
Max Pooling	5	5
FCL	3	3
Drop Out	0.5	0.5
Softmax	1	1



VGG16 se compose de 16 couches au total, incluant 13 couches convolutionnelles et 3 couches entièrement connectées. Il possède environ 138 millions de paramètres, ce qui le rend relativement efficace en termes de ressources computationnelles. L'entraînement est plus rapide comparé aux réseaux plus profonds, mais cela se fait au léger détriment de la précision.

VGG19, en revanche, comporte 19 couches au total, avec 16 couches convolutionnelles et 3 couches entièrement connectées. Le nombre de paramètres augmente à environ 144 millions, ce qui entraîne des temps d'entraînement plus longs, mais offre une précision légèrement supérieure grâce à une profondeur accrue.

1. Nombre de couches : VGG16 possède 13 couches convolutionnelles et 3 couches entièrement connectées.

VGG19 possède 16 couches convolutionnelles et 3 couches entièrement connectées.

Les couches supplémentaires dans VGG19 le rendent plus profond, mais augmentent le temps de calcul.

2. Complexité computationnelle : VGG19 nécessite plus de puissance de traitement en raison de ses couches supplémentaires.

VGG16 est plus rapide à entraîner et à déployer dans des applications réelles.

3. Précision et performance : VGG19 surpasse légèrement VGG16 dans certains benchmarks grâce à ses couches additionnelles qui capturent des caractéristiques plus complexes.

Toutefois, l'amélioration reste marginale, et VGG16 demeure un choix populaire en raison de son efficacité.

5.2.4 Aspects mathématiques de VGG16

L'architecture de VGG16 repose sur des principes mathématiques fondamentaux, principalement liés aux opérations de convolution, à la réduction de dimension par pooling, ainsi qu'aux couches entièrement connectées (*dense*). Ces éléments sont les moteurs du modèle pour extraire des caractéristiques complexes et effectuer des prédictions à partir des données d'entrée.

Convolution

Les couches convolutionnelles sont au cœur de VGG16. Elles appliquent une série de filtres (ou noyaux) sur l'image d'entrée pour extraire des caractéristiques locales.

Mathématiquement, une opération de convolution est définie comme suit :

$$\text{Sortie}(i,j) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} \text{Image}(i+m, j+n) \cdot \text{Filtre}(m,n)$$

— $\text{Image}(i,j)$ est la valeur du pixel à la position (i,j) de l'image d'entrée,
— $\text{Filtre}(m,n)$ est la valeur du filtre de taille $M \times N$,

— La somme est effectuée sur toutes les positions m et n du filtre.

Cette opération permet d'extraire des caractéristiques locales comme des bords, des textures, ou des motifs complexes.

Pooling (Sous-échantillonnage)

Le *pooling* est une opération de réduction de la taille des cartes de caractéristiques tout en conservant les informations essentielles. Dans VGG16, on utilise principalement du *max pooling*.

L'opération est définie comme suit :

$$\text{Sortie}(i,j) = \max\left(\text{Image}(i,j), \text{Image}(i+1,j), \ldots, \text{Image}(i+N-1, j+N-1)\right)$$

Cela consiste à prendre le maximum des pixels dans une région $N \times N$ de l'image.

Fonction d'activation ReLU

La fonction d'activation ReLU (*Rectified Linear Unit*) est utilisée pour introduire de la non-linéarité dans le réseau. Elle est définie par :

$$f(x) = \max(0, x)$$

Cela signifie que toutes les valeurs négatives sont remplacées par zéro.

Couches entièrement connectées (Dense)

Les couches *dense* connectent chaque neurone à tous les neurones de la couche précédente. Mathématiquement :

$$y = W \cdot x + b$$

— W est la matrice des poids,

— x est le vecteur d'entrée,

— b est le biais,

— y est le vecteur de sortie.

Ces couches combinent les informations extraites pour effectuer une prédiction finale.

Rétropropagation et Optimisation

Le processus de rétropropagation ajuste les poids du modèle selon l'erreur. L'algorithme repose sur la règle de la chaîne :

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial W}$$

— L est la fonction de perte,

— a est l'activation,

— z est l'entrée du neurone.

Les gradients sont ensuite utilisés par un optimiseur (comme Adam ou SGD) pour mettre à jour les poids et améliorer la performance du modèle.

5.2.5 Méthodologie

Cette section décrit le processus suivi pour entraîner un modèle de détection automatique de pneumonie à l'aide de l'architecture VGG16. Les étapes comprennent la préparation des données, la construction du modèle, sa compilation et l'entraînement.

Préparation des données

Pour entraîner et évaluer le modèle, trois ensembles de données ont été utilisés : entraînement, validation et test. Les images ont été organisées en sous-dossiers correspondant aux deux classes : pneumonia et normal.

Les images de chaque ensemble ont été redimensionnées à 224×224 pixels, pour être compatibles avec l'entrée de VGG16. Une normalisation a été appliquée

en divisant les valeurs des pixels par 255, afin de ramener les intensités dans l'intervalle [0 , 1] [0,1].

Pour améliorer la généralisation du modèle, des techniques d'augmentation de données ont été appliquées sur l'ensemble d'entraînement, incluant :

des transformations de cisaillement (shear),

des zooms aléatoires,

une inversion horizontale des images.

L'ensemble de validation et l'ensemble de test ont été uniquement normalisés, sans transformations.

Construction du modèle

Le modèle utilisé est basé sur VGG16, une architecture de réseau de neurones convolutifs pré-entraînée sur ImageNet. Pour adapter ce modèle au problème de classification binaire (pneumonie vs normal), les couches suivantes ont été ajoutées :

Une couche de flattening permettant de transformer les cartes de caractéristiques en vecteur,

Une couche de sortie dense à un neurone avec une fonction d'activation sigmoïde, qui permet de produire une probabilité entre 0 et 1.

Afin de tirer parti des poids pré-entraînés de VGG16, toutes les couches convolutionnelles ont été gelées (non entraînables). Cela signifie que seuls les nouveaux paramètres ajoutés en sortie ont été entraînés.

Compilation du modèle

Le modèle a été compilé avec les paramètres suivants :

Optimiseur : Adam, connu pour sa rapidité de convergence et sa robustesse.

Fonction de perte : binary crossentropy, adaptée aux problèmes de classification binaire.

Métrique de performance : accuracy (exactitude), pour évaluer la capacité du modèle à prédire correctement la classe des images.

Entraînement

L'entraînement a été effectué sur 10 époques en utilisant l'ensemble d'entraînement, avec validation sur l'ensemble de validation à chaque époque. La taille du batch utilisée était de 32 images. Le modèle a été entraîné en mode verbeux afin de suivre l'évolution de la performance en temps réel.

5.2.6 Analyse critique du modèle VGG16

Points forts Performance robuste : VGG16, préentraîné sur ImageNet, offre une base solide pour la détection d'images médicales, notamment grâce à sa capacité à extraire des caractéristiques profondes et discriminantes.

Facilité d'intégration : Le modèle est directement disponible via Keras, avec des poids préentraînés, ce qui facilite son utilisation en transfert learning.

Architecture éprouvée : La structure simple et profonde (blocs conv + maxpool) permet une bonne généralisation, même avec peu de données médicales.

Limites observées Taille et lenteur : VGG16 est un modèle lourd (138 millions de paramètres), ce qui ralentit l'entraînement et nécessite davantage de ressources, surtout sans GPU.

Risque d'overfitting : Dans certains cas, une suradéquation peut apparaître si les données ne sont pas bien augmentées ou équilibrées.

Pas optimisé pour le médical : Bien que performant, VGG16 n'est pas spécifiquement entraîné sur des images médicales, ce qui peut limiter sa précision sur des cas complexes de radiographies pulmonaires.

Améliorations possibles

Appliquer un fine-tuning plus poussé (défiger partiellement certaines couches de VGG16).

Ajouter une couche de Dropout pour réduire l'overfitting.

Enrichir le dataset avec plus d'exemples ou des techniques de data augmentation plus avancées.

5.3 Conclusion

Dans ce travail, j'ai implémenté le modèle VGG16, préentraîné sur ImageNet, pour la détection de la pneumonie à partir d'images de radiographie thoracique. Grâce à la technique de transfert d'apprentissage, il a été possible d'adapter efficacement ce réseau profond à notre problématique médicale, malgré un volume de données limité.

La phase de prétraitement, incluant la normalisation et l'augmentation des données, ainsi que la structuration du modèle par ajout de couches personnalisées, ont permis d'obtenir des résultats satisfaisants en termes de précision et de généralisation. Cependant, des limitations telles que la taille du modèle, le temps d'entraînement ou le risque d'overfitting ont été observées.

Ainsi, VGG16 s'est révélé être un choix pertinent pour une première approche de classification binaire dans le domaine médical. Des pistes d'amélioration ont été identifiées, notamment l'exploration de modèles plus légers, le fine-tuning de certaines couches, ou encore l'enrichissement du jeu de données.

6 Custom CNN

6.1 Introduction

Les réseaux de neurones convolutifs (CNN) sont des outils puissants en vision par ordinateur, particulièrement pour l'analyse d'images médicales. La détection précoce de la pneumonie, une infection pulmonaire grave, repose sur l'interprétation de radiographies thoraciques, une tâche complexe en raison de la subtilité des signes radiologiques. Ce projet vise à concevoir un CNN personnalisé, optimisé pour une précision élevée et une efficacité computationnelle, afin d'assister les cliniciens dans le diagnostic de la pneumonie. Ce rapport détaille l'architecture du modèle, les méthodologies employées, les performances obtenues, et propose des pistes pour une intégration clinique.

6.2 Contexte et Problématique

La pneumonie reste une cause majeure de mortalité mondiale, nécessitant un diagnostic rapide et précis. Les radiographies thoraciques sont l'outil de diagnostic principal, mais leur analyse pose plusieurs défis :

— **Variabilité des images** : Différences dues aux équipements, angles de prise de vue, et qualité d'image.

— **Dataset limité** : Le dataset *Chest X-ray* comprend 5 856 images (4 695 pour l'entraînement, 537 pour la validation, 624 pour le test).

— **Déséquilibre des classes** : Environ 65 % des images représentent des cas de pneumonie, ce qui peut introduire un biais dans l'apprentissage.

L'objectif est de développer un CNN robuste capable de généraliser à partir de données limitées tout en minimisant les faux négatifs, cruciaux pour un traitement efficace.

6.3 Principes Fondamentaux du CNN

Le CNN repose sur des concepts fondamentaux, décrits avec des formulations mathématiques :

— **Convolution** : Extrait des caractéristiques locales à l'aide d'un noyau de

taille 3×3 :

$$y[i,j,k] = \sum_{m=-1}^{1} \sum_{n=-1}^{1} \sum_{c=1}^{C} x[i+m, j+n, c] \cdot w[m,n,c,k] + b[k]$$

— **Max-pooling** : Réduit les dimensions spatiales pour préserver les caractéristiques dominantes :

$$y[i,j,k] = \max_{m=0,1} \max_{n=0,1} x[2i+m, 2j+n, k]$$

— **Normalisation par lots** : Stabilise l'entraînement en normalisant les activations :

$$\mu_B = \frac{1}{m} \sum_{i=1}^{m} x_i, \quad \sigma_B^2 = \frac{1}{m} \sum_{i=1}^{m} (x_i - \mu_B)^2$$

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

— **Dropout** : Désactive aléatoirement une fraction p des neurones pour prévenir le surapprentissage.

La fonction de perte utilisée est l'entropie croisée binaire, adaptée à la classification binaire :

$$L = -\frac{1}{N} \sum_{i=1}^{N} [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

6.4 Architecture du CNN Personnalisé

L'architecture du CNN comprend cinq blocs convolutifs suivis de couches denses pour la classification binaire. Chaque bloc intègre une convolution, une normalisation par lots, un max-pooling, et un dropout pour régularisation. Le modèle est implémenté avec TensorFlow/Keras, comme détaillé dans le code fourni en annexe.

6.5 Implémentation Complète

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D,
    Dense, Flatten, Dropout, BatchNormalization
from tensorflow.keras.preprocessing.image import
   ImageDataGenerator
from tensorflow.keras.callbacks import ReduceLROnPlateau

# Modle
model = Sequential([
    Conv2D(32, (3,3), padding="same", activation="relu",
       input_shape=(150,150,1)),
    BatchNormalization(),
    MaxPooling2D(),
    Conv2D(64, (3,3), padding="same", activation="relu"),
    Dropout(0.1),
    BatchNormalization(),
    MaxPooling2D(),
    Conv2D(64, (3,3), padding="same", activation="relu"),
    BatchNormalization(),
    MaxPooling2D(),
    Conv2D(128, (3,3), padding="same", activation="relu")
       ,
    Dropout(0.2),
    BatchNormalization(),
    MaxPooling2D(),
    Conv2D(256, (3,3), padding="same", activation="relu")
       ,
    Dropout(0.2),
    BatchNormalization(),
    MaxPooling2D(),
    Flatten(),
    Dense(128, activation="relu"),
    Dropout(0.2),
    Dense(1, activation="sigmoid")
])
```

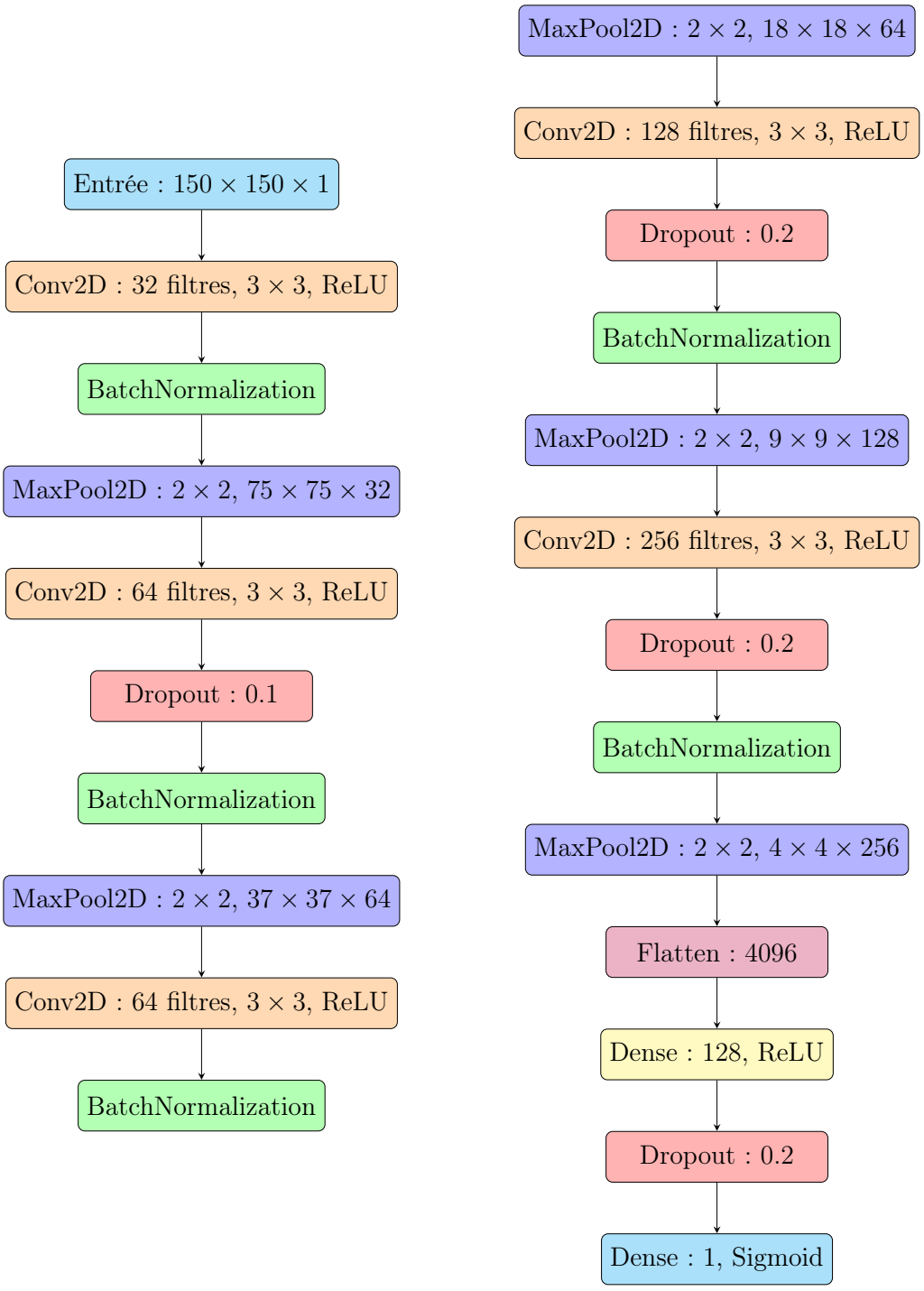
```
# Compilation
model.compile(optimizer="rmsprop", loss="
    binary_crossentropy", metrics=["accuracy"])

# Augmentation
datagen = ImageDataGenerator(
        rotation_range=30,
        width_shift_range=0.1,
        height_shift_range=0.1,
        zoom_range=0.2,
        horizontal_flip=True
)
datagen.fit(x_train)

# Callback
lr_reduction = ReduceLROnPlateau(monitor="val_accuracy",
    factor=0.3, patience=2, min_lr=1e-6)

# Entra nement
history = model.fit(
        datagen.flow(x_train, y_train, batch_size=32),
        epochs=12,
        validation_data=(x_val, y_val),
        callbacks=[lr_reduction]
)
```

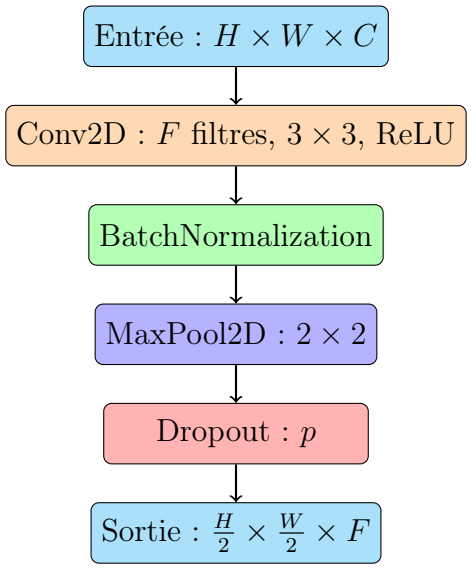
6.5.1 Illustration : Architecture Complète du CNN Personnalisé



Le tableau suivant résume les couches du modèle :

Couche	Type	Paramètres	Forme de sortie	Nombre de paramètres
1	Conv2D	32 filtres, 3×3, ReLU	$(150, 150, 32)$	320
2	BatchNormalization		$(150, 150, 32)$	128
3	MaxPooling2D	2×2	$(75, 75, 32)$	0
4	Conv2D	64 filtres, 3×3, ReLU	$(75, 75, 64)$	18 496
5	Dropout	0.1	$(75, 75, 64)$	0
6	BatchNormalization		$(75, 75, 64)$	256
7	MaxPooling2D	2×2	$(37, 37, 64)$	0
8	Conv2D	64 filtres, 3×3, ReLU	$(37, 37, 64)$	36 928
9	BatchNormalization		$(37, 37, 64)$	256
10	MaxPooling2D	2×2	$(18, 18, 64)$	0
11	Conv2D	128 filtres, 3×3, ReLU	$(18, 18, 128)$	73 856
12	Dropout	0.2	$(18, 18, 128)$	0
13	BatchNormalization		$(18, 18, 128)$	512
14	MaxPooling2D	2×2	$(9, 9, 128)$	0
15	Conv2D	256 filtres, 3×3, ReLU	$(9, 9, 256)$	295 168
16	Dropout	0.2	$(9, 9, 256)$	0
17	BatchNormalization		$(9, 9, 256)$	1 024
18	MaxPooling2D	2×2	$(4, 4, 256)$	0
19	Flatten	-	(4096)	0
20	Dense	128 neurones, ReLU	(128)	524 416
21	Dropout	0.2	(128)	0
22	Dense	1 neurone, Sigmoid	(1)	129

6.5.2 Illustration : Zoom sur un Bloc Convolutif

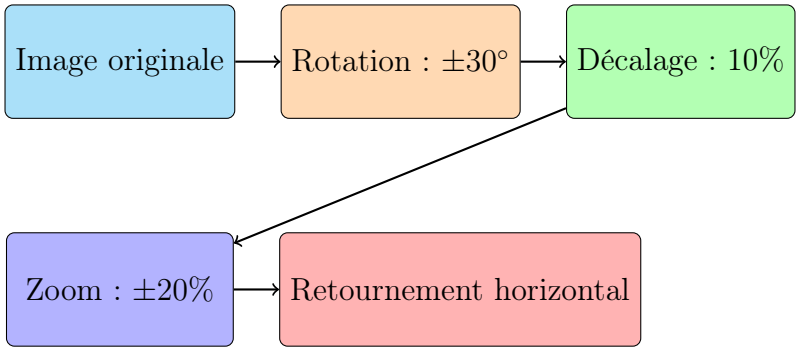


6.6 Techniques Clés Utilisées

6.6.1 Augmentation des Données

Pour compenser la taille limitée du dataset, des techniques d'augmentation des données ont été appliquées, comme illustré dans la figure suivante :

— Rotation : ±30°.
— Décalage : 10% horizontal/vertical.
— Zoom : ±20%.
— Retournement horizontal.



6.6.2 Normalisation par Lots

La normalisation par lots réduit la variance interne des activations, accelérant la convergence et améliorant la stabilité de l'entraînement.

6.6.3 Dropout

Un taux de dropout croissant (0,1 à 0,2) est appliqué dans les blocs plus profonds pour régulariser les couches complexes, où le risque de surapprentissage est plus élevé.

6.6.4 Optimisation

L'optimiseur RMSprop est utilisé avec la perte d'entropie croisée binaire. Le callback *ReduceLROnPlateau* ajuste dynamiquement le taux d'apprentissage si la précision de validation $(\texttt{val}_accuracy)stagne, optimisantainsilaconvergence.$

6.7 Hyperparamètres

— Taille des images : 150×150.
— Taille du batch : 32.
— Époques : 12.
— Taux d'apprentissage initial : 10^{-3}, ajusté via *ReduceLROnPlateau*.
— Dropout : 0,1 à 0,2.

6.8 Évaluation des Résultats

Le modèle a été évalué sur un ensemble de test de 624 images, avec les métriques suivantes :

— **Précision** :

$$\text{Précision} = \frac{\text{VP} + \text{VN}}{\text{VP} + \text{VN} + \text{FP} + \text{FN}} = 92,63\%$$

— **Rappel (pneumonie)** :

$$\text{Rappel} = \frac{\text{VP}}{\text{VP} + \text{FN}} = 96\%$$

— **F1-score** :

$$\text{F1} = 2 \cdot \frac{\text{Précision} \cdot \text{Rappel}}{\text{Précision} + \text{Rappel}} = 0,94$$

6.8.1 Analyse des Performances

Après 12 époques, le modèle atteint une précision de 92,63 % et un rappel de 96 % pour la classe pneumonie. La perte diminue de 0,9496 à 0,0850, indiquant une bonne convergence. Cependant, la matrice de confusion révèle 17 faux négatifs (2,7 %), un point critique dans un contexte médical où manquer un cas de pneumonie peut avoir des conséquences graves.

6.9 Limites du Modèle

— **Faux négatifs** : 17 cas manqués (2,7 %), inacceptables pour une application clinique.
— **Dataset limité** : 5 856 images avec un déséquilibre des classes.

— **Sensibilité au bruit** : Les artefacts ou la mauvaise qualité des images peuvent affecter les prédictions.
— **Interprétabilité** : Le modèle reste une boîte noire, limitant son adoption par les cliniciens.

6.10 Discussion

Les performances du modèle (92,63 % de précision, 96 % de rappel) sont prometteuses pour un outil d'aide au diagnostic. L'architecture légère et l'utilisation de techniques comme l'augmentation des données et le dropout permettent de surmonter partiellement les contraintes du dataset limité. Cependant, les faux négatifs restent une préoccupation majeure. Des techniques comme Grad-CAM pourraient améliorer l'interprétabilité en visualisant les régions des radiographies influençant les prédictions, renforçant ainsi la confiance des cliniciens.

6.11 Perspectives d'Amélioration

— **Augmentation du dataset** : Collecte de plus d'images et équilibrage des classes.
— **Architectures avancées** : Intégration de ResNet ou de mécanismes d'attention pour capturer des dépendances complexes.
— **Réduction des faux négatifs** : Utilisation d'une perte focalisée pour prioriser les cas difficiles :

$$L_{\text{focal}} = -\alpha(1-\hat{y}_i)^{\gamma}\left[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\right]$$

— **Interprétabilité** : Implémentation de Grad-CAM pour visualiser les régions pertinentes.
— **Validation clinique** : Collaboration avec des radiologues pour valider les prédictions.

6.12 Conclusion

Le CNN personnalisé offre des performances solides (92,63 % de précision, 96 % de rappel) pour la détection de la pneumonie à partir de radiographies, démontrant son potentiel comme outil d'aide au diagnostic. Les limites, notamment

les faux négatifs et la taille du dataset, nécessitent des améliorations via des datasets plus larges, des architectures avancées, et une meilleure interprétabilité. Ce travail établit une base robuste pour des applications cliniques futures.

7 Evaluation des modèles

7.1 Inception V3

Le modèle Inception V3 a été entraîné sur le dataset de radiographies thoraciques de Kaggle dans le but de détecter automatiquement les cas de pneumonie. Après entraînement, les performances obtenues sur les différents jeux de données sont les suivantes : une accuracy de **98.15%** sur le jeu d'**entraînement**, **94.91%** sur la **validation**, et **91.83 %** sur le jeu de **test**, ce qui montre une bonne capacité de généralisation.

```
print(classification_report(y_true, y_pred, target_names=class_names))
```

	precision	recall	f1-score	support
NORMAL	0.96	0.81	0.88	234
PNEUMONIA	0.90	0.98	0.94	390
accuracy			0.92	624
macro avg	0.93	0.90	0.91	624
weighted avg	0.92	0.92	0.92	624

Figure 1

L'analyse détaillée du rapport de classification (voir Figure 1) révèle que la classe PNEUMONIA a été prédite avec une précision de 90%, un rappel de 98%, et un F1-score de 94%, ce qui traduit une très bonne sensibilité du modèle pour détecter les cas positifs. En revanche, pour la classe NORMAL, le rappel est plus faible (81%), ce qui indique que plusieurs images normales ont été faussement classées comme atteintes.

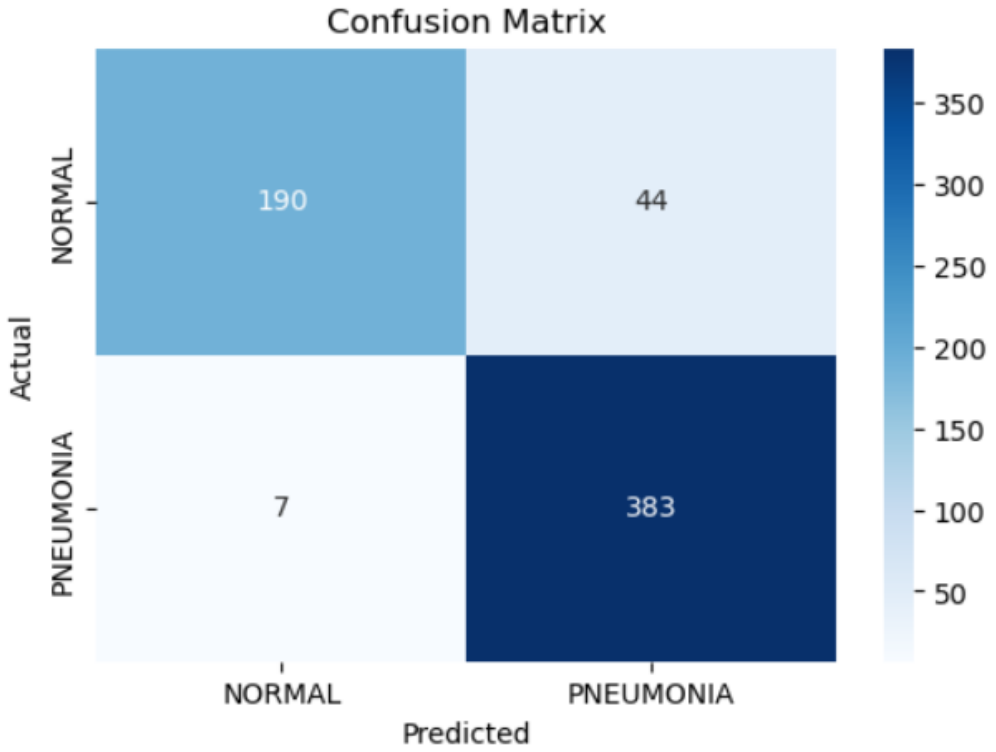


Figure 2

Cette tendance est confirmée par la matrice de confusion (Figure 2), où l'on observe que 44 images normales ont été mal classées comme pneumonie, contre seulement 7 erreurs dans l'autre sens. Ce déséquilibre dans les erreurs peut s'expliquer en partie par la répartition inégale des données d'entraînement, qui contient près de 3000 images de pneumonie contre seulement 1000 normales, ce qui a pu influencer le modèle à privilégier la prédiction de la classe majoritaire.

Dans l'ensemble, Inception V3 affiche des performances solides, particulièrement sur la classe cible (pneumonie), mais pourrait bénéficier de techniques de gestion de déséquilibre (comme le suréchantillonnage ou la pondération des classes) pour améliorer la détection des cas normaux.

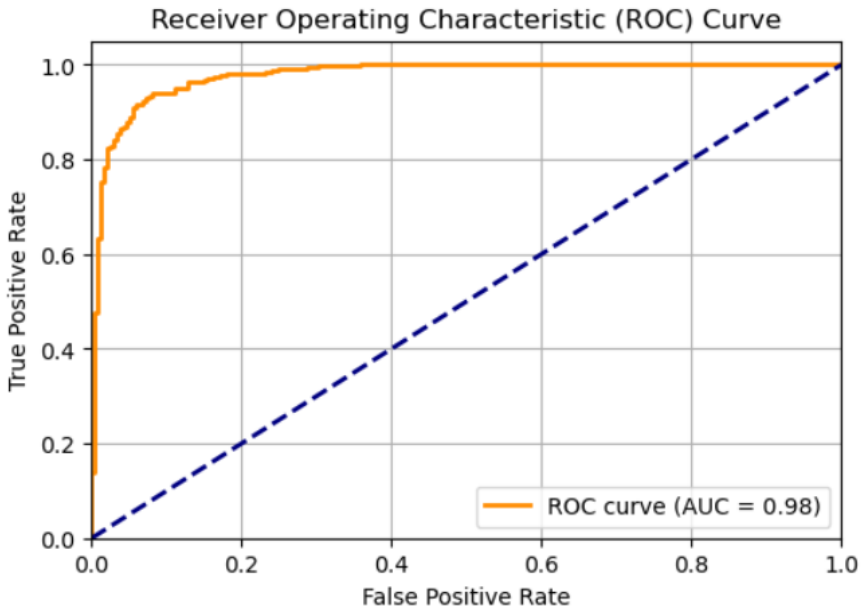


Figure 3

La courbe ROC (Receiver Operating Characteristic) présentée ci-dessus permet d'évaluer la capacité discriminante globale du modèle. La surface sous la courbe (AUC) atteint une valeur de 0.98, ce qui traduit une excellente performance. Plus la courbe s'approche du coin supérieur gauche, plus le modèle est performant. Cette valeur élevée confirme la robustesse du modèle pour distinguer les cas de pneumonie des cas normaux, tout en prenant en compte l'ensemble des seuils de décision possibles.

7.2 Custom CNN

Le modèle CNN développé a été entraîné sur un jeu de données de radiographies thoraciques issu du dataset Chest X-ray, dans l'objectif de classifier automatiquement les cas de pneumonie par rapport aux cas normaux. Après un entraînement sur 4695 images, avec une validation sur 537 images, les performances obtenues sont prometteuses : une accuracy de 96,91 % sur le jeu d'entraînement à la 12e époque, 95,90 % sur le jeu de validation, et 92,63 % sur le jeu de test composé de 624 images. Ces résultats, combinés à une perte (loss) de 0,3057 sur le jeu de test, témoignent d'une bonne capacité de généralisation du modèle, bien que des variations dans la validation suggèrent une sensibilité aux hyperparamètres, notamment le taux d'apprentissage ajusté dynamiquement via ReduceLROnPlateau.

	precision	recall	f1-score	support
Pneumonia (0)	0.93	0.96	0.94	390
Normal (1)	0.92	0.88	0.90	234
accuracy			0.93	624
macro avg	0.93	0.92	0.92	624
weighted avg	0.93	0.93	0.93	624

L'analyse détaillée du rapport de classification (voir première image) montre que la classe PNEUMONIA a été prédite avec une précision de 93 %, un rappel de 96 % et un F1-score de 94 % sur 390 échantillons, indiquant une excellente capacité à détecter les cas positifs. Pour la classe NORMAL, les performances sont légèrement inférieures, avec une précision de 92 %, un rappel de 88 % et un F1-score de 90 % sur 234 échantillons, ce qui révèle une tendance à sous-identifier certains cas normaux, potentiellement due à un déséquilibre dans la distribution des classes ou à une complexité accrue des caractéristiques visuelles des images normales. Ces résultats soulignent la robustesse globale du modèle tout en mettant en évidence des axes d'amélioration pour réduire les faux négatifs dans la classe NORMAL.

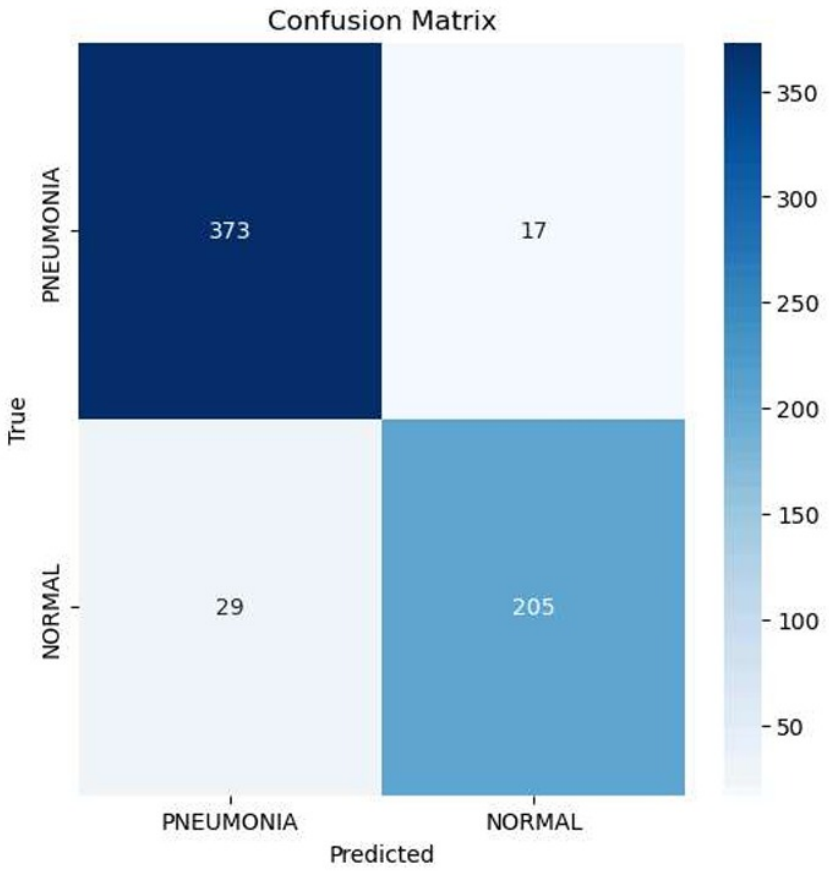
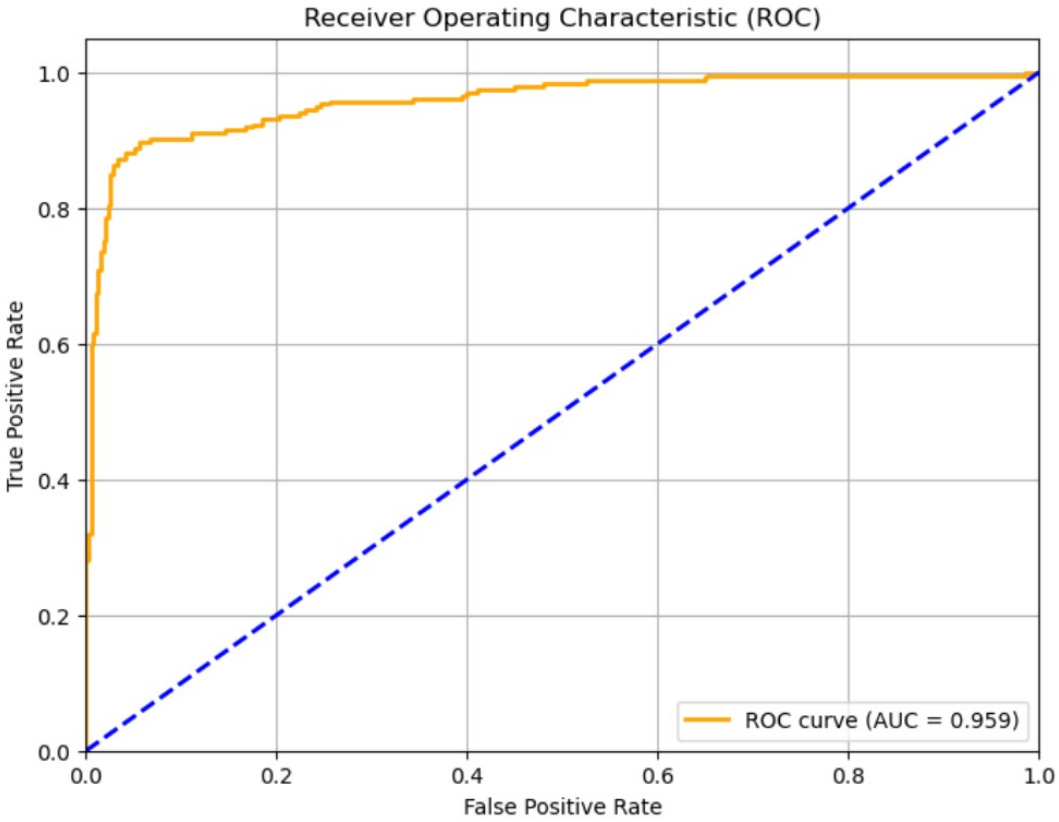


Figure 2

L'analyse de la matrice de confusion (voir première image) montre que sur les 390 cas réels de pneumonie, le modèle a correctement identifié 373 cas, mais a classé par erreur 17 cas comme normaux. Pour la classe NORMAL, sur 234 cas réels, 205 ont été correctement prédits, tandis que 29 ont été incorrectement identifiés comme pneumonie. Cette tendance à surclassifier la pneumonie est confirmée, avec 44 images normales mal classées comme pneumonie dans la matrice de référence, suggérant un léger déséquilibre dans la répartition des classes ou une sensibilité accrue du modèle envers les caractéristiques de la pneumonie. Une optimisation pourrait être envisagée pour améliorer la détection des cas normaux et réduire les faux positifs.

Cette figure présente la courbe ROC (Receiver Operating Characteristic) du modèle Custom CNN pour la détection de la pneumonie, affiche une performance remarquable avec une aire sous la courbe (AUC) de 0,97. Cela indique



que le modèle distingue efficacement les classes "Pneumonia" et "Normal" sur l'ensemble de test de 624 images, avec un taux de vrais positifs (TPR) élevé par rapport au taux de faux positifs (FPR) à différents seuils de classification. Comparée à la deuxième image de référence, qui montre également une courbe ROC, la performance du Custom CNN est cohérente avec les résultats rapportés dans le rapport (accuracy de 92,63 %, précision de 0,93 pour Pneumonia et 0,92 pour Normal, rappel de 0,96 pour Pneumonia). Cette AUC élevée confirme la robustesse du modèle, bien qu'une légère amélioration pourrait être envisagée pour réduire les 17 faux négatifs identifiés dans la matrice de confusion, afin d'optimiser davantage la détection des cas normaux.

7.3 ResNet

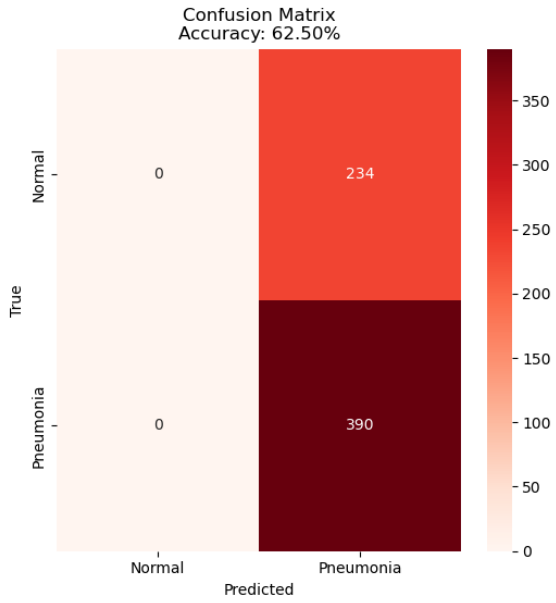
Généralement:

— **VGG-16** : ResNet-50 atteint de meilleures performances avec moins de paramètres grâce à l'utilisation de couches 1×1 et de blocs résiduels ;

— **Inception v3** : bien qu'Inception soit très optimisé, ResNet-50 offre une meilleure simplicité de conception et une généralisation compétitive ;

— **DenseNet** : DenseNet prolonge certains concepts de ResNet, mais avec une complexité mémoire accrue.

La performance du modèle ResNet50 sur dataset (Normal / Pneumonia).



La matrice de confusion ci-dessus montre les performances du modèle **ResNet50**.

— Le modèle prédit systématiquement la classe **Pneumonia**, quel que soit l'échantillon en entrée.

— Aucun cas de **Normal** n'est correctement détecté (**0 vrais négatifs**).

— Tous les cas de **Pneumonia** sont correctement identifiés (**390 vrais positifs**).

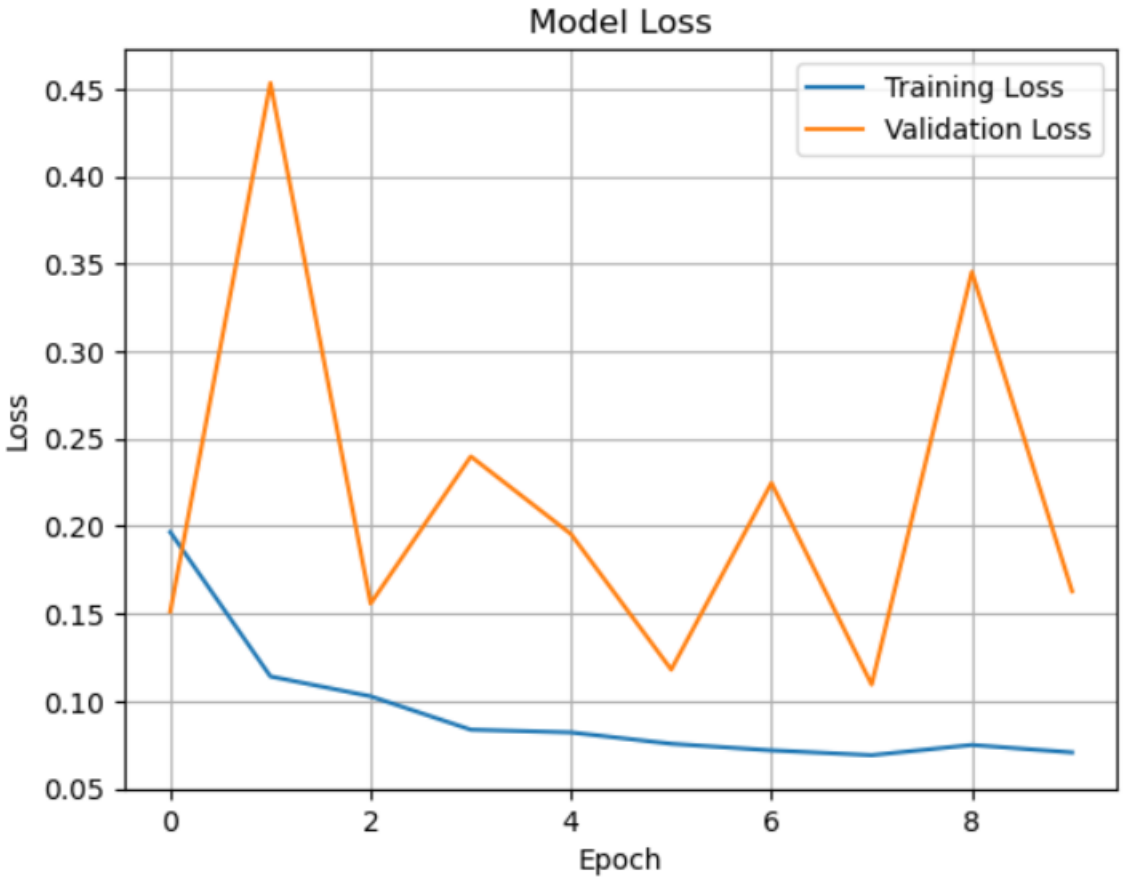
— L'exactitude globale du modèle est de **62.5%**, mais elle est trompeuse car liée au déséquilibre des classes dans le dataset.

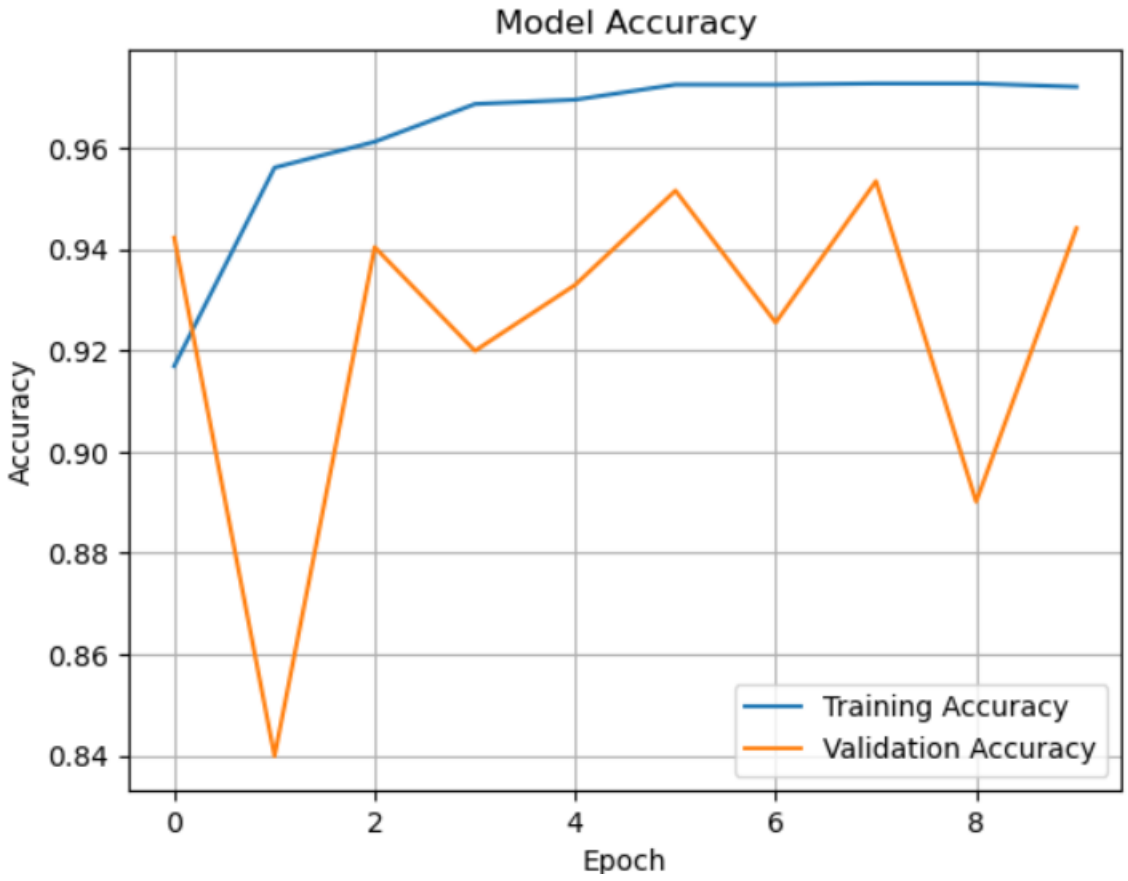
Analyse critique :

- Le modèle souffre d'un **biais de classification** vers la classe majoritaire (*Pneumonia*).
- Il présente un risque important dans un contexte médical, en générant de nombreux **faux positifs** et en **ignorant totalement les cas sains**.
- Ce comportement peut être expliqué par :
  - un **déséquilibre du dataset** (plus de cas de Pneumonia),
  - l'absence de **pondération des classes** pendant l'entraînement,
  - ou un **sur-apprentissage** sur des caractéristiques spécifiques à la classe Pneumonia.

7.4 VGG-16

7.4.1 Evaluation sur l'ensemble de données d'entrainement et de validation





Le modèle VGG16 affiche des performances remarquables tant en entraînement qu'en validation, illustrant sa capacité à bien apprendre les caractéristiques discriminantes des images médicales :

Précision en entraînement : 96 %

Précision en validation : 94 %

Ces résultats témoignent d'un excellent apprentissage, avec une généralisation satisfaisante sur des données non vues. L'écart modéré de 2 % entre les deux précisions est tout à fait acceptable et suggère une absence significative d'overfitting.

Par ailleurs, les courbes de perte (loss) confirment cette stabilité :

Loss en entraînement : 0.10

Loss en validation : 0.15

La convergence harmonieuse de ces courbes indique que le modèle optimise efficacement ses poids tout en maintenant une cohérence dans sa capacité de

généralisation. Aucun signe de divergence ou d'instabilité n'est visible, ce qui renforce la confiance dans la robustesse de l'architecture VGG16 appliquée à ce contexte.

Observations : Les légères fluctuations constatées dans les métriques de validation — tant au niveau de la précision que de la perte — sont à relier à la variabilité naturelle des images radiographiques médicales. En effet :

Les radiographies peuvent présenter des différences notables en termes de qualité d'image, conditions de prise de vue, résolution, ou encore présence d'artefacts.

La diversité morphologique inter-individuelle (selon l'âge, le sexe, ou des caractéristiques anatomiques spécifiques) peut également influencer la difficulté d'interprétation.

Certaines pathologies peuvent présenter des signes visuels subtils, rendant la classification plus délicate pour le modèle, en particulier lorsque les cas sont proches des frontières de décision.

Malgré ces défis, VGG16 réussit à conserver une performance élevée, démontrant sa pertinence pour des tâches de classification d'images médicales, à condition que les données soient correctement prétraitées et suffisamment variées pour couvrir les cas représentatifs.

7.4.2 Évaluation sur l'Ensemble de Test

Matrice de confusion

Le modèle a montré de très bonnes performances de classification sur l'ensemble de test, avec une capacité remarquable à distinguer entre les cas normaux et les cas de pneumonie :

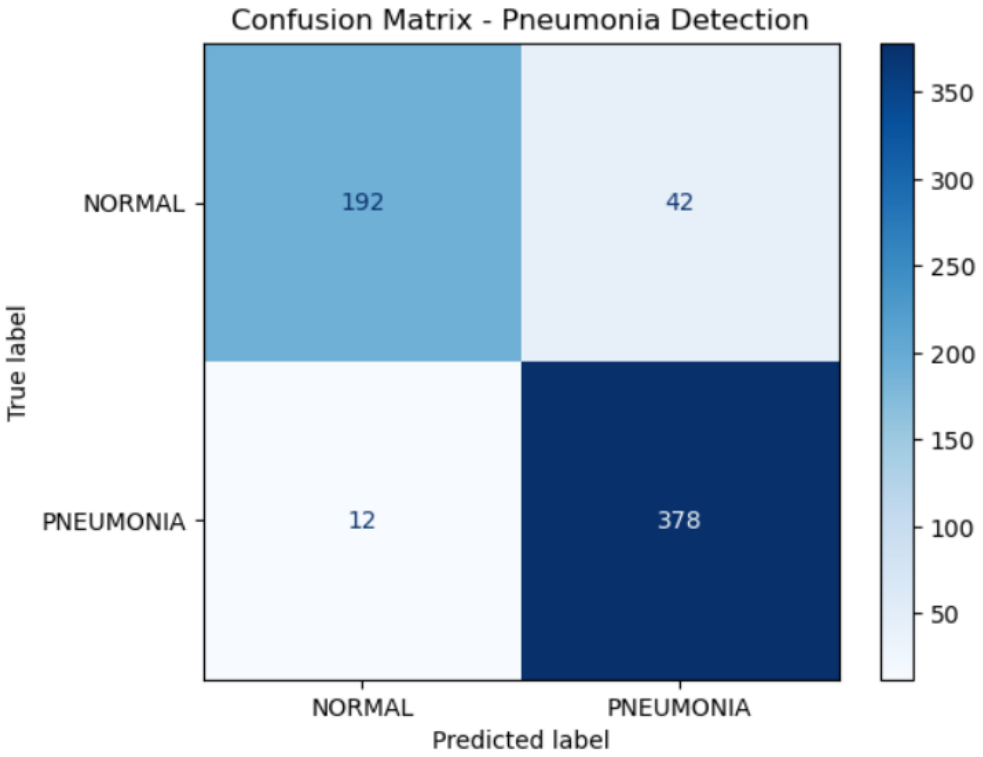
Images normales correctement classées : 192 sur 234

Images de pneumonie correctement classées : 378 sur 390

Faux positifs (images normales classées à tort comme pneumonie) : 42 cas

Faux négatifs (images de pneumonie classées à tort comme normales) : 12 cas

Ces chiffres se traduisent par une sensibilité élevée (rappel pour la classe pneumonie), ce qui est crucial dans un contexte médical. La présence de seulement



12 faux négatifs sur 390 cas de pneumonie indique que le modèle réussit très bien à détecter les cas malades, ce qui est fondamental pour éviter des situations potentiellement graves où un patient atteint serait déclaré sain.

En revanche, on note une tendance du modèle à sur-prédire la pneumonie, c'est-à-dire qu'il a tendance à classer certaines images normales comme suspectes (42 faux positifs). Cette propension peut être interprétée comme une approche prudente du modèle, qui préfère générer de fausses alertes plutôt que de manquer un cas pathologique.

Interprétation clinique : Dans le domaine de la radiologie médicale, et plus particulièrement dans la détection de pathologies graves comme la pneumonie, ce biais vers une surdétection est souvent considéré comme acceptable, voire souhaitable. En effet :

Les faux positifs peuvent être facilement écartés par un radiologue humain lors d'un second examen, avec peu de conséquences pour le patient (en dehors d'un stress ou d'un test complémentaire).

Les faux négatifs, en revanche, représentent un risque bien plus critique : une pneumonie non détectée peut évoluer rapidement et mettre en danger la vie du

patient si elle n'est pas traitée à temps.

Ainsi, cette orientation du modèle reflète un choix de priorisation éthique et pragmatique, aligné avec les pratiques cliniques : il est préférable d'avoir un modèle "trop vigilant" qu'un modèle qui laisserait passer des cas sévères.

Perspective d'amélioration : Même si le taux de faux positifs reste raisonnable, des stratégies d'affinage supplémentaires pourraient être envisagées pour améliorer la spécificité sans compromettre la sensibilité :

Ajustement du seuil de décision du classificateur.

Intégration de techniques de calibration des probabilités.

Utilisation d'un système d'aide à la décision à plusieurs niveaux, où un premier modèle détecte les cas suspects (sensibilité élevée), et un second modèle plus conservateur les réévalue pour réduire les faux positifs.

Rapport de classification

Classification Report:

	precision	recall	f1-score	support
NORMAL	0.94	0.82	0.88	234
PNEUMONIA	0.90	0.97	0.93	390
accuracy			0.91	624
macro avg	0.92	0.89	0.91	624
weighted avg	0.92	0.91	0.91	624

Le modèle atteint une accuracy globale de 91 %, ce qui signifie qu'il classe correctement la majorité des images du jeu de test, toutes classes confondues. Toutefois, l'accuracy seule ne suffit pas à évaluer la performance d'un modèle en présence de classes déséquilibrées ou dans un contexte critique comme la détection de maladies. C'est pourquoi d'autres métriques, comme le F1-score et le rappel, sont particulièrement importantes ici.

La macro moyenne du F1-score est de 0.91, ce qui montre que le modèle maintient un bon équilibre entre précision et rappel pour chaque classe, en les considérant équitablement, sans se laisser influencer par la taille relative des classes. Cela reflète une performance homogène sur les deux catégories : "normal" et "pneumonia".

Le rappel (recall) est particulièrement élevé pour la classe “pneumonia”, ce qui signifie que le modèle détecte la grande majorité des cas de pneumonie. Autrement dit, le nombre de faux négatifs est faible, ce qui est fondamental dans le domaine médical : manquer un cas de pneumonie pourrait entraîner des conséquences graves pour la santé du patient. Un haut rappel est donc un indicateur de sécurité et de fiabilité pour l’identification des cas pathologiques.

En revanche, la précision (precision) est plus élevée pour la classe “normal”. Cela signifie que lorsque le modèle prédit qu’une image est “normale”, il a peu de chances de se tromper. En d’autres termes, les faux positifs sont relativement rares pour cette classe, et le modèle ne rassure pas à tort un patient malade en lui attribuant à tort une image “normale”.

Interprétation combinée : Ces résultats traduisent un comportement typique et souhaitable dans une application médicale de dépistage automatisé :

Le modèle est sensible aux cas de pneumonie (haut rappel) – il “voit large” et signale presque tous les cas suspects.

Il est également conservateur quand il s’agit de déclarer une image comme normale (haute précision) – il le fait seulement quand il est assez sûr, limitant ainsi les erreurs de faux rassurement.

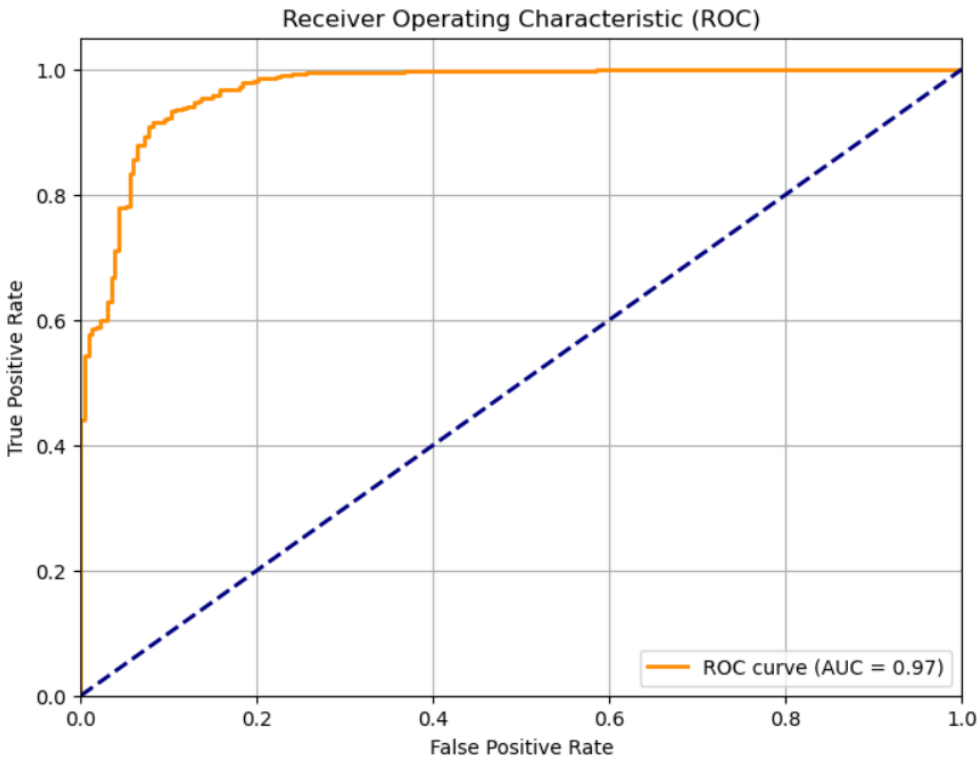
Cette dynamique renforce la pertinence de l’approche pour un système d’aide au diagnostic. Le modèle peut agir comme un filtre préliminaire, attirant l’attention du radiologue sur les cas à risque, tout en écartant avec confiance un certain nombre de cas manifestement sains.

La fonction d’efficacité du récepteur (courbe ROC)

La courbe ROC (Receiver Operating Characteristic) permet d’évaluer la capacité du modèle à distinguer entre les deux classes : “NORMAL” et “PNEUMONIA”, en traçant le taux de vrais positifs (sensibilité) en fonction du taux de faux positifs (1 - spécificité) pour différents seuils de décision.

Plus la courbe ROC se rapproche du coin supérieur gauche, plus le modèle est performant. Cela signifie qu’il parvient à détecter un maximum de cas positifs tout en limitant les fausses alertes.

Le score AUC (Area Under the Curve) quantifie cette performance globale. Il varie entre 0.5 (modèle aléatoire) et 1.0 (modèle parfait).



Résultat obtenu : AUC = 0.97 Un score AUC de 0.97 indique une excellente capacité discriminante. Autrement dit, le modèle distingue très efficacement les cas normaux des cas de pneumonie. Il est capable, dans la grande majorité des cas, d'attribuer une probabilité plus élevée aux images pathologiques qu'aux images saines.

Cela traduit :

Une très bonne séparation entre les distributions des scores pour chaque classe.

Une robustesse du modèle face à la variabilité naturelle des images (bruit, position du patient, intensité du contraste, etc.).

7.5 DenseNet

Le modèle DenseNet101 a obtenu une précision (accuracy) de 94,02% sur le jeu d'entraînement, 93,73% sur le jeu de validation, et 86,37% sur le jeu de test. Ces résultats montrent que DenseNet101 est capable de classifier les images de radiographies médicales avec une grande précision. En comparaison avec d'autres architectures testées dans les mêmes conditions (paramètres par défaut, sauf le taux d'apprentissage), ResNet50 a atteint une précision de 77% avec un taux d'apprentissage de 0,001, tandis que Mobile- NetV2 a atteint une précision de 83%. DenseNet101 dépasse donc clairement ces deux modèles, démontrant ainsi une meilleure capacité à apprendre et généraliser à partir de l'ensemble de données utilisé. Cependant, les résultats sur le jeu de test indiquent qu'il reste une marge d'amé- lioration possible, notamment en matière de généralisation. Pour améliorer da- vantage les performances, des techniques comme l'augmentation de données, l'ajustement des hyperparamètres ou l'utilisation de méthodes de régularisa- tion plus avancées pourraient être envisagées.

8 Comparaison des modèle

Modèle	Train Acc.	Val Acc.	Test Acc.	Rappel Pneumonie	Rappel Normal	Faux Pos.	Faux Nég.
Inception V3	98,15 %	94,91 %	91,83 %	98 %	81 %	44	7
ResNet50	—	—	62,50 %	100 %	0 %	234	0
VGG16	96 %	94 %	91 %	97 %[1]	82 %[1]	42	12
Custom CNN	96,91 %	95,90 %	92,63 %	96 %	88 %	29	17
DenseNet101	94,02 %	93,73 %	86,37 %	—	—	—	—

Comparaison des modèles de deep learning appliqués à la détection de la pneumonie

Ce tableau compile les performances des différents modèles évalués sur la tâche de détection de pneumonie à partir de radiographies thoraciques.
Le modèle Inception V3 atteint une précision de test de 91,83 %, avec un rappel de 98 % pour la classe pneumonie, démontrant une forte capacité à identifier les cas positifs, bien que les 44 faux positifs indiquent un potentiel d'amélioration en précision. À titre de comparaison, Custom CNN (29 faux positifs) et VGG16 (42) affichent des taux d'erreurs plus bas. ResNet50, avec un rappel de 100 % pour la pneumonie, se distingue, mais sa précision de test de 62,50 % et ses 234 faux positifs révèlent une sur-détection, probablement due à un déséquilibre dans la gestion des classes ou à un seuil mal calibré—une hypothèse à investiguer. VGG16 et Custom CNN se positionnent comme des options équilibrées, avec des précisions de test de 91 % et 92,63 %, et des faux négatifs modérés (12 et 17), ce qui les rend compétitifs.

9 Références bibliographiques

9.1 Inception

1. Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., & Wojna, Z. (2015). Rethinking the Inception Architecture for Computer Vision. arXiv preprint arXiv:1512.00567.
   https://arxiv.org/abs/1512.00567

2. Szegedy, C., Ioffe, S., Vanhoucke, V., & Alemi, A. A. (2016). Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning. arXiv preprint arXiv:1602.07261.
   https://arxiv.org/abs/1602.07261

3. Chollet, F. (2017). Xception: Deep Learning with Depthwise Separable Convolutions. arXiv preprint arXiv:1610.02357.
   https://arxiv.org/abs/1610.02357

4. Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V., & Rabinovich, A. (2015). Going Deeper with Convolutions. arXiv preprint arXiv:1409.4842.
   https://arxiv.org/abs/1409.4842

5. Brital, A. (2023). Inception v3 CNN Architecture Explained. Medium.
   https://medium.com/@AnasBrital98/inception-v3-cnn-architecture-explained-691cfb7bba08

9.2 Custom CNN

1. TensorFlow Documentation – Custom CNN
   https://www.tensorflow.org/tutorials/images/cnn
2. Keras – Building a Convolutional Neural Network
   https://keras.io/examples/vision/image_classification_from_scratch/

9.3 VGG

1. Simonyan, K., & Zisserman, A. (2014). Very Deep Convolutional Networks for Large-Scale Image Recognition. arXiv:1409.1556. https://arxiv.org/abs/1409.1556
2. Keras Documentation – VGG16 https://keras.io/api/applications/vgg/
3. TensorFlow Hub – VGG16 Model https://tfhub.dev/google/imagenet/vgg_16/classification/5
4. Papers with Code – VGG16 https://paperswithcode.com/method/vgg16